



ORACLE

Data Engineer Masterclass

Oracle Database In-Memory

Database In-Memory Product Management

Oracle Corporation

Blog: blogs.oracle.com/in-memory



What is Database In-Memory?



Row Format Databases vs. Column Format Databases

Rows Stored
Contiguously



- Transactions run faster on row format
 - Example: Query or Insert a sales order
 - Fast processing few rows, many columns

Columns
Stored
Contiguously



- Analytics run faster on column format
 - Example : Report on sales totals by region
 - Fast accessing few columns, many rows

Until Now Must Choose One Format and Suffer Tradeoffs

Introducing Database In-Memory

What's your favorite data format?

Row Format

*Fast for OLTP!
Slower for Analytics*



Column Format

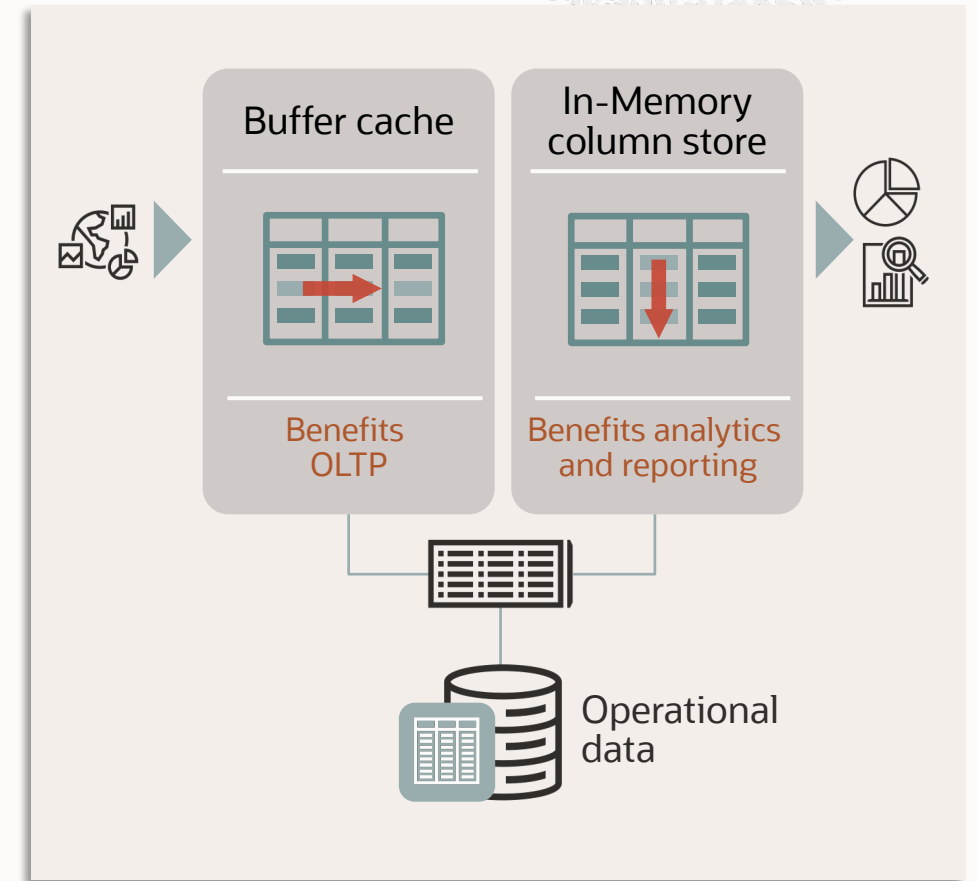
*Fast for Analytics!
REALLY slow for OLTP!*



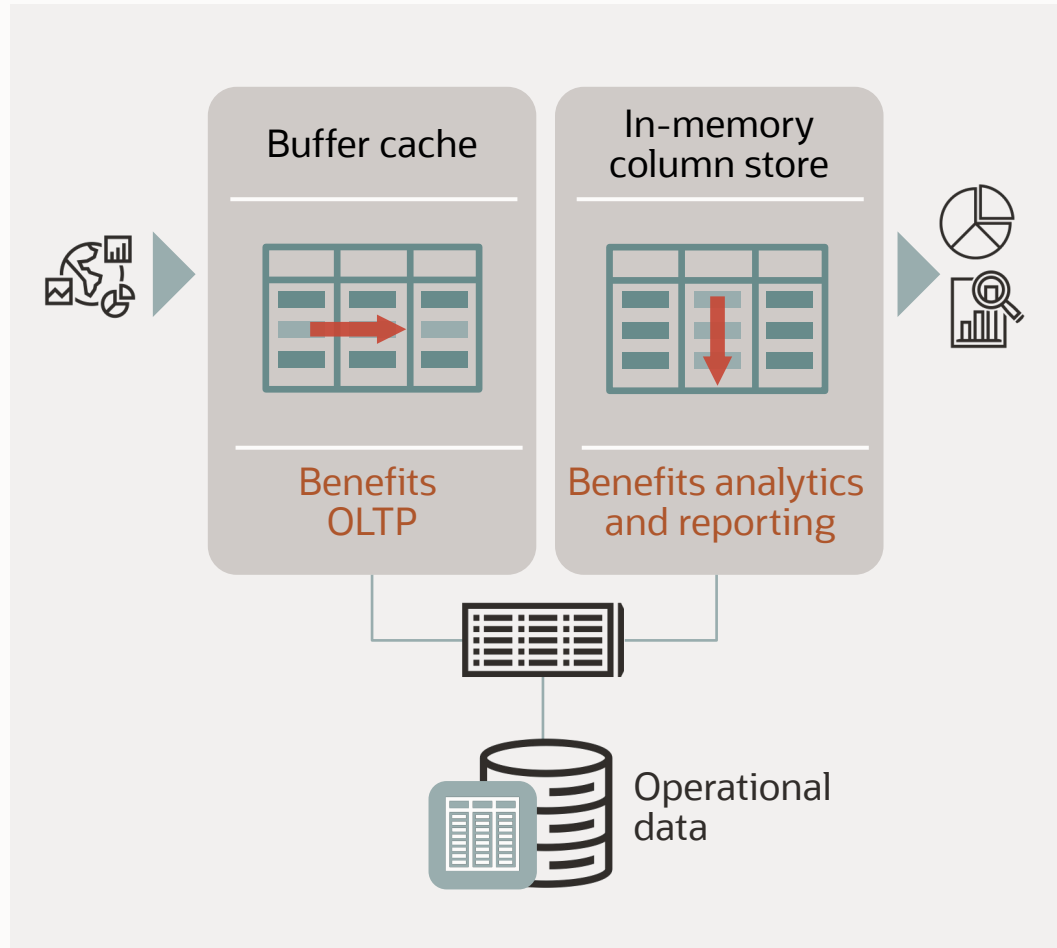
(+)

Dual Format

*Best for both
Fast Analytics
and fast OLTP
(No need for
Analytic indexes)*



Breakthrough: Dual Format Database



- **BOTH** row and column formats for same table
- **Simultaneously active and transactionally consistent**
- Analytics & reporting use new in-memory Column format
- OLTP uses proven row format
- No application changes required

Oracle In-Memory: Simple to Implement

1. Configure Memory Capacity

```
inmemory_size = XXX GB
```

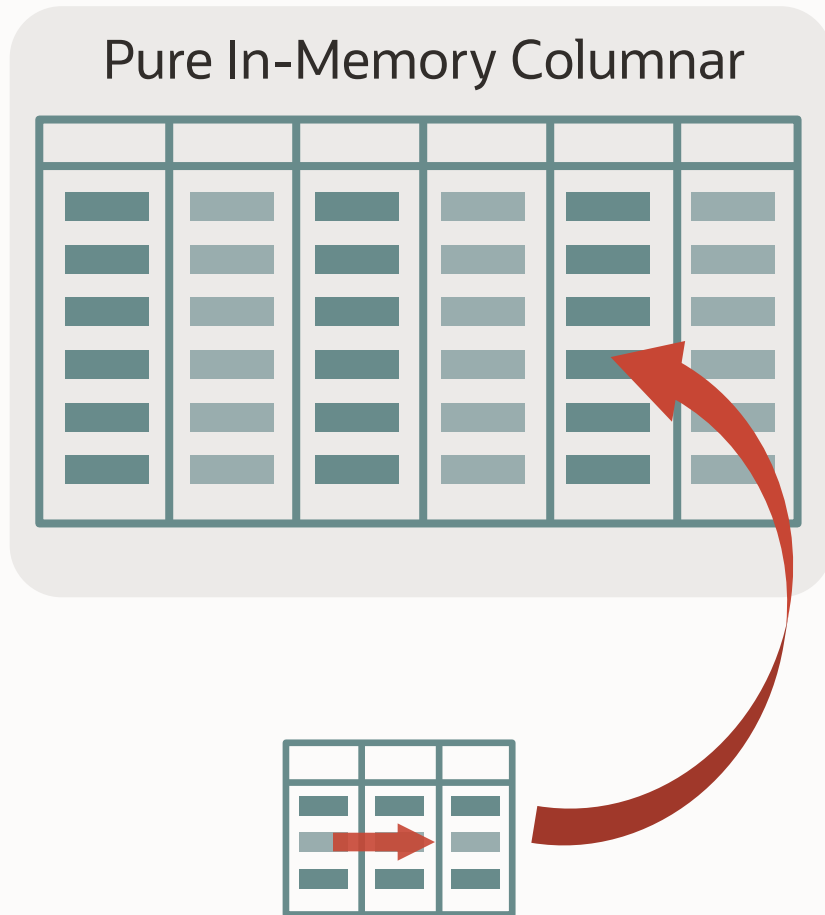
2. Configure tables or partitions to be in memory

```
alter table | partition ... inmemory;
```

3. Later drop analytic indexes to speed up OLTP

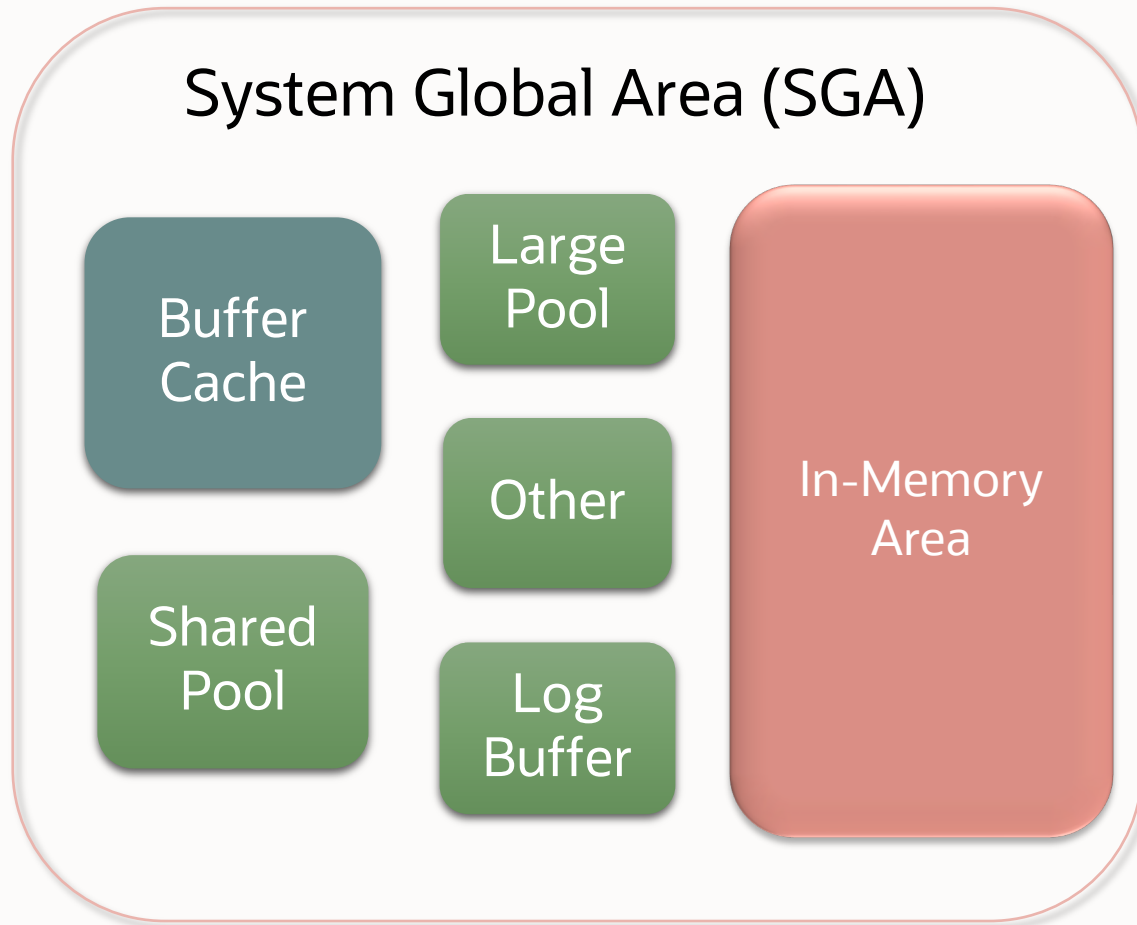
How Does Database In-Memory Work?

Oracle In-Memory Columnar Technology



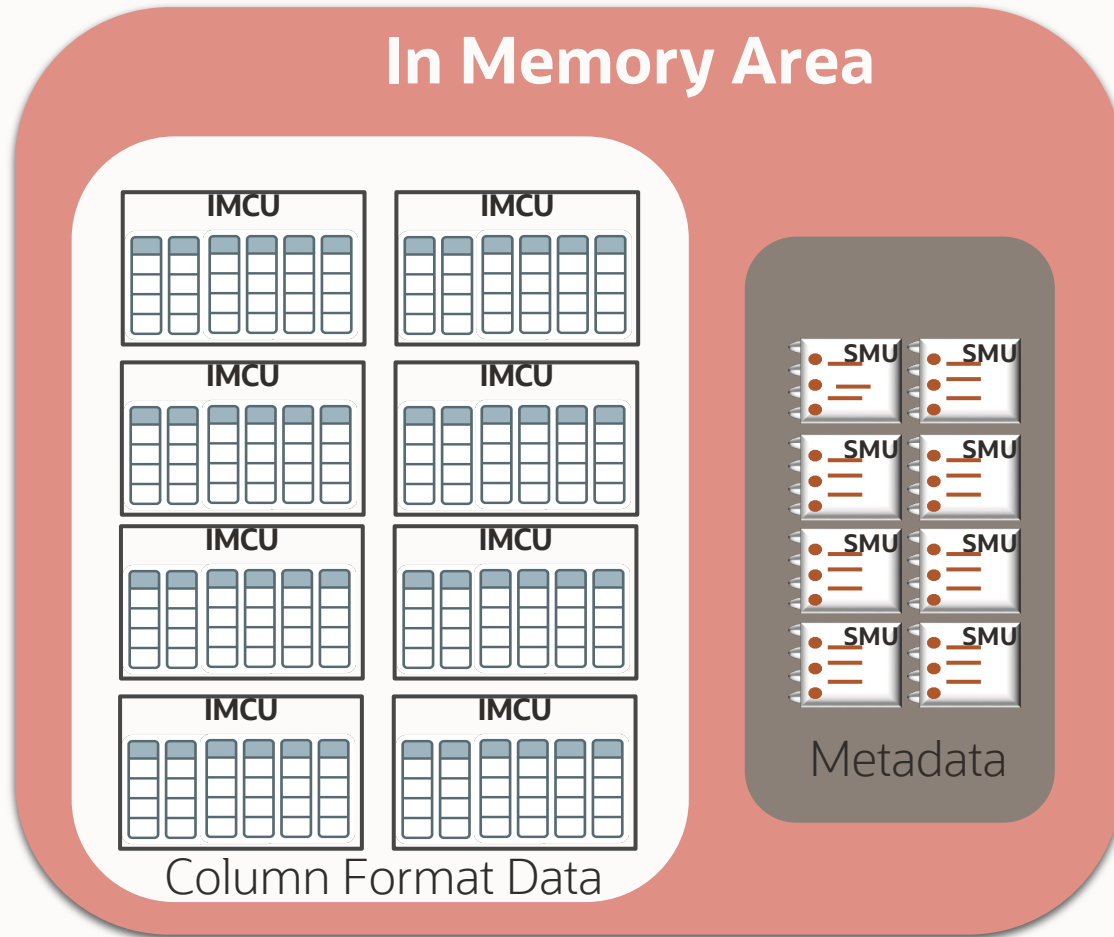
- Pure in-memory columnar format
 - Not persistent, so no undo or redo is generated
- Can be enabled for table, partition, subpartition or materialized view
- 2x to 20x compression typical
- Available on all hardware platforms
- Does not require the whole database to be in-memory
 - Can be enabled for just active data (table, partition, sub-partition, materialized view)

In-Memory Area: **Static Area within SGA**



- Contains data in the new In-Memory Columnar Format
- Controlled by INMEMORY_SIZE parameter
 - Minimum size of 100MB
- Can be re-sized larger while database is running (12.2)
- SGA_TARGET must be large enough to accommodate In-Memory area

Composition of In-Memory Area



- Contains two subpools:
 - IMCU pool: Stores In Memory Compression Units (IMCUs)
 - SMU pool: Stores Snapshot Metadata Units (SMUs)
- IMCUs contain column formatted data
- SMUs contain metadata and transactional information

View In-Memory Area Usage



V\$INMEMORY_AREA:
Current size of pools in
the In-Memory area

```
SQL> SELECT * FROM v$inmemory_area;
```

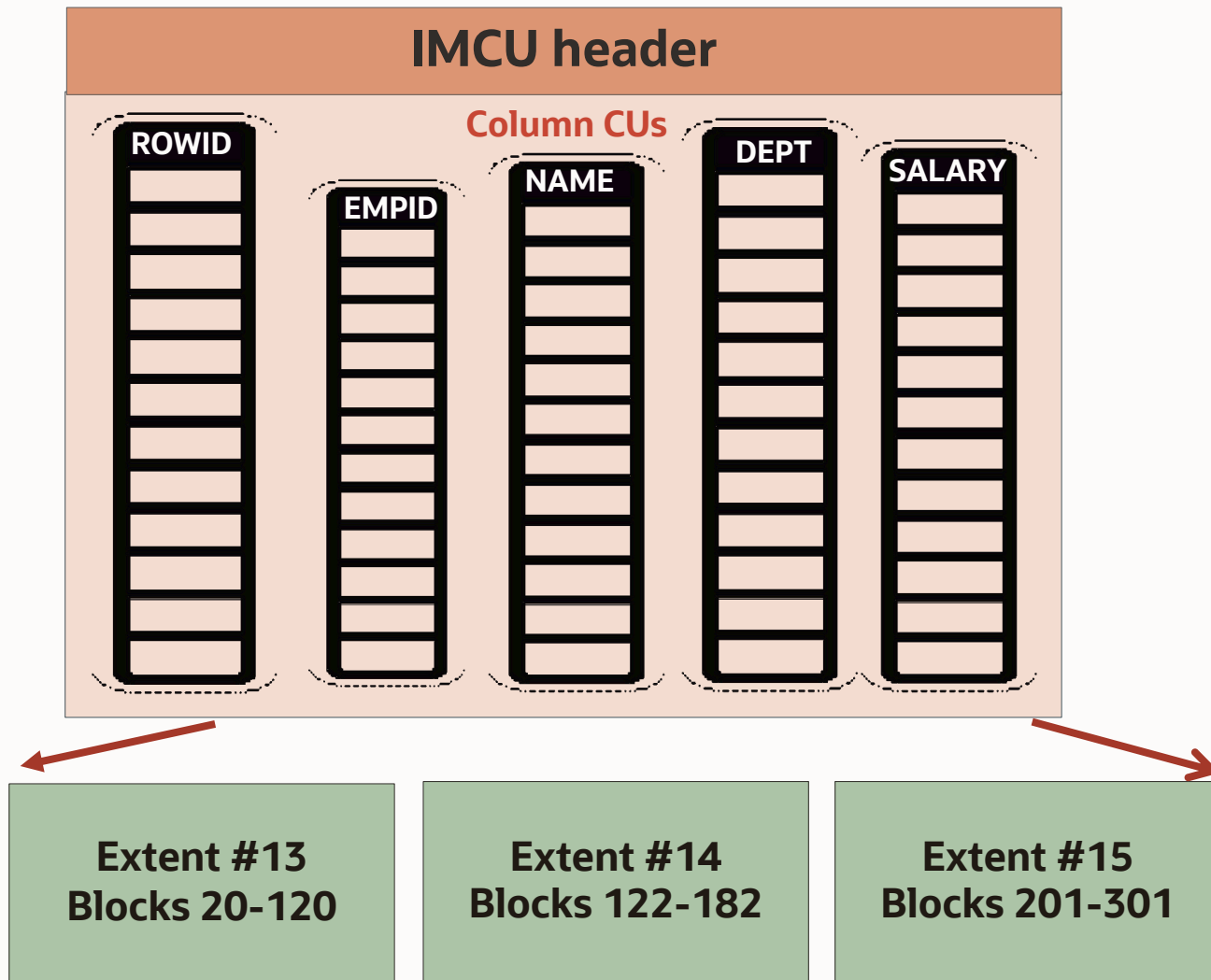
POOL	ALLOC_BYTES	USED_BYTES	POPULATE_STATUS
-----	-----	-----	-----
1MB POOL	5,179,965,440	3,241,148,416	DONE
64KB POOL	570,425,344	9,568,256	DONE

V\$IM_SEGMENTS:
List of segments
currently populated in
the In-Memory column
store

```
SQL> SELECT owner, segment_name, populate_status,  
inmemory_size, bytes_not_populated  
FROM v$im_segments;
```

OWNER	NAME	STATUS	In-Memory Size	Bytes Not Populated
-----	-----	-----	-----	-----
SSB	LINEORDER	COMPLETED	3,206,086,656	0
SSB	DATE_DIM	COMPLETED	1,179,648	0
SSB	SUPPLIER	COMPLETED	2,228,224	0
SSB	PART	COMPLETED	18,022,400	0
SSB	CUSTOMER	COMPLETED	23,199,744	0

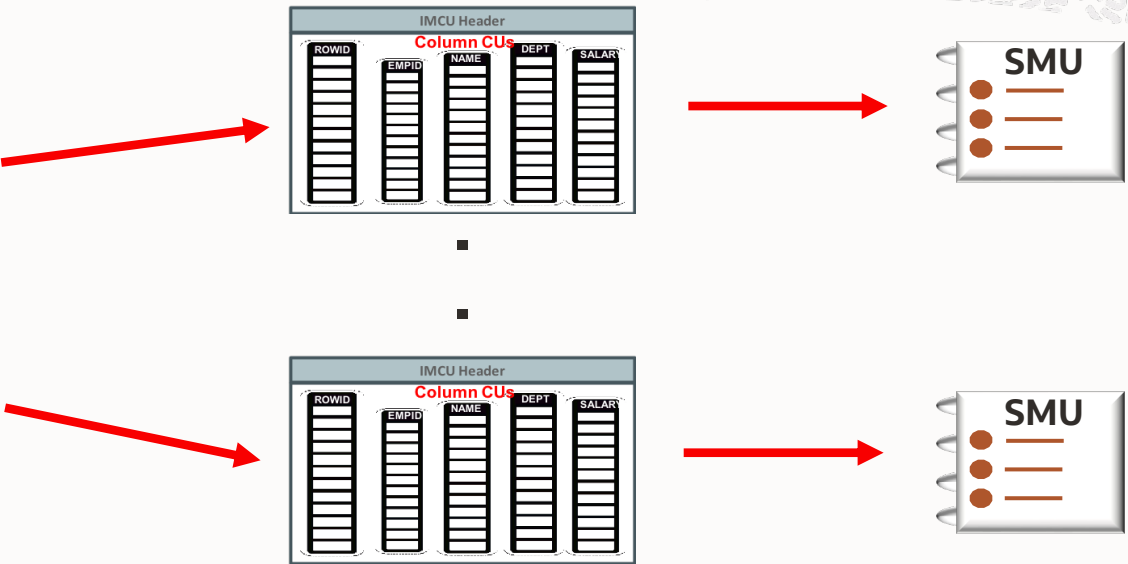
Composition of In-Memory Compression Unit (IMCU)



- Unit of column store allocation
 - Columnar representation of a large number of rows from an object
 - Rows from one or more table extents
- Actual size depends on size of rows, compression factor, etc.
- Each column stored as a separate contiguous **Column Compression Unit** (column CU)
 - Rowids also stored as a Column CU

Composition of In-Memory Object

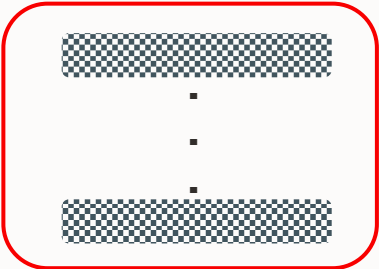
Employee Table		



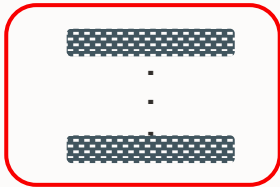
An IMCU/SMU is composed of one or more 1MB/64KB memory segments

An object in memory is composed of one or more IMCUs/SMUs

In-Memory Area
(1MB Pool)

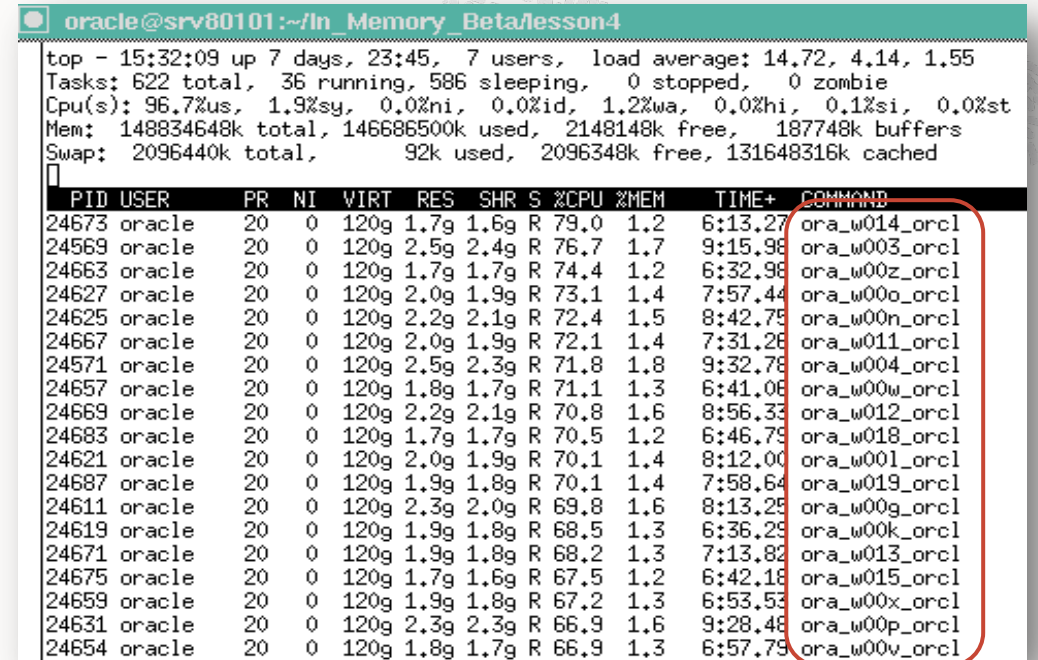


In-Memory Area
(64 KB Pool)



Population

- Order in which objects are populated controlled by PRIORITY subclause:
 - **ALTER TABLE sales**
INMEMORY PRIORITY HIGH;
- Levels:
 - CRITICAL > HIGH > MEDIUM > LOW
 - Controls order (not speed) of populate
- Default PRIORITY is NONE
 - Populate only on first access



```
oracle@srv80101:~/ln_Memory_Beta/lesson4
top - 15:32:09 up 7 days, 23:45, 7 users, load average: 14.72, 4.14, 1.55
Tasks: 622 total, 36 running, 586 sleeping, 0 stopped, 0 zombie
Cpu(s): 96.7%us, 1.9%sy, 0.0%ni, 0.0%id, 1.2%wa, 0.0%hi, 0.1%si, 0.0%st
Mem: 148834648k total, 146686500k used, 2148148k free, 187748k buffers
Swap: 2096440k total, 92k used, 2096348k free, 131648316k cached

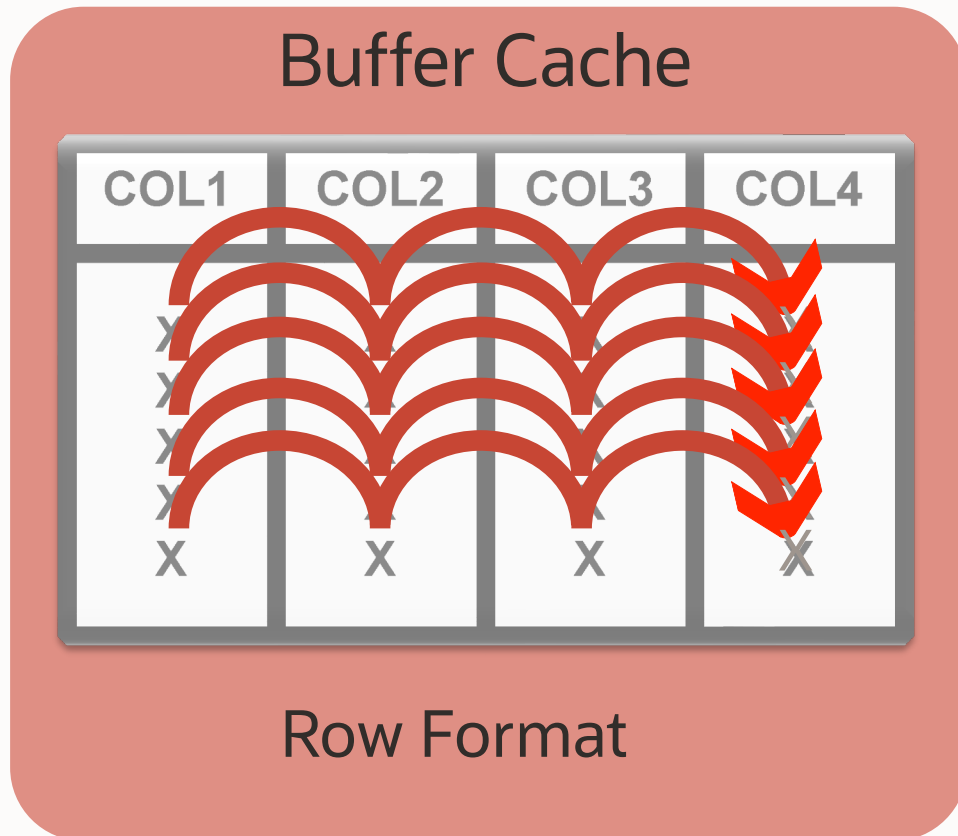
  PID USER      PR  NI  VIRT  RES  SHR  S %CPU  %MEM    TIME+  COMMAND
 24673 oracle    20   0 120g  1.7g  1.6g  R 79.0   1.2   6:13.27 ora_w014_orcl
 24569 oracle    20   0 120g  2.5g  2.4g  R 76.7   1.7   9:15.98 ora_w003_orcl
 24663 oracle    20   0 120g  1.7g  1.7g  R 74.4   1.2   6:32.98 ora_w00z_orcl
 24627 oracle    20   0 120g  2.0g  1.9g  R 73.1   1.4   7:57.44 ora_w00a_orcl
 24625 oracle    20   0 120g  2.2g  2.1g  R 72.4   1.5   8:42.75 ora_w00n_orcl
 24667 oracle    20   0 120g  2.0g  1.9g  R 72.1   1.4   7:31.26 ora_w011_orcl
 24571 oracle    20   0 120g  2.5g  2.3g  R 71.8   1.8   9:32.78 ora_w004_orcl
 24657 oracle    20   0 120g  1.8g  1.7g  R 71.1   1.3   6:41.06 ora_w00w_orcl
 24669 oracle    20   0 120g  2.2g  2.1g  R 70.8   1.6   8:56.33 ora_w012_orcl
 24683 oracle    20   0 120g  1.7g  1.7g  R 70.5   1.2   6:46.75 ora_w018_orcl
 24621 oracle    20   0 120g  2.0g  1.9g  R 70.1   1.4   8:12.00 ora_w001_orcl
 24687 oracle    20   0 120g  1.9g  1.8g  R 70.1   1.4   7:58.64 ora_w019_orcl
 24611 oracle    20   0 120g  2.3g  2.0g  R 69.8   1.6   8:13.25 ora_w00g_orcl
 24619 oracle    20   0 120g  1.9g  1.8g  R 68.5   1.3   6:36.25 ora_w00k_orcl
 24671 oracle    20   0 120g  1.9g  1.8g  R 68.2   1.3   7:13.82 ora_w013_orcl
 24675 oracle    20   0 120g  1.7g  1.6g  R 67.5   1.2   6:42.18 ora_w015_orcl
 24659 oracle    20   0 120g  1.9g  1.8g  R 67.2   1.3   6:53.53 ora_w00x_orcl
 24631 oracle    20   0 120g  2.3g  2.3g  R 66.9   1.6   9:28.48 ora_w00p_orcl
 24654 oracle    20   0 120g  1.8g  1.7g  R 66.9   1.3   6:57.79 ora_w00v_orcl
```

- Population completed by background processes
ora_w00x_orcl
- Number of processes controlled by parameter:

INMEMORY_MAX_POPULATE_SERVERS

Why is an In-Memory Scan Faster?

Querying Row Data



```
SELECT COL4 FROM MYTABLE;
```



RESULT

Querying Columnar Data

IM Column Store

COL1	COL2	COL3	COL4
X	X	X	X
X	X	X	X
X	X	X	X
X	X	X	X
X	X	X	X

Column Format

SELECT **COL4** FROM MYTABLE;



RESULT

Why Not Cache The Table In The Buffer Cache?

Compare Column-store to Row-store Query

```
SQL> -- In-Memory Column Store query
SQL>
SQL> select max(lo_ordtotalprice) most_expensive_order From LINEORDER;
```

MOST_EXPENSIVE_ORDER

57346348

Elapsed: 00:00:00.01

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT				5401 (100)	
1	SORT AGGREGATE		1	6		
2	TABLE ACCESS INMEMORY FULL	LINEORDER	59M	343M	5401 (16)	00:00:01

```
SQL> -- Buffer Cache query with the column store disabled via NO_INMEMORY hint
SQL>
SQL> select /*+ NO_INMEMORY */ max(lo_ordtotalprice) most_expensive_order From LINEORDER;
```

MOST_EXPENSIVE_ORDER

57346348

Elapsed: 00:00:08.38

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT				123K(100)	
1	SORT AGGREGATE		1	6		
2	TABLE ACCESS FULL	LINEORDER	59M	343M	123K (1)	00:00:05

Database In-Memory Technology

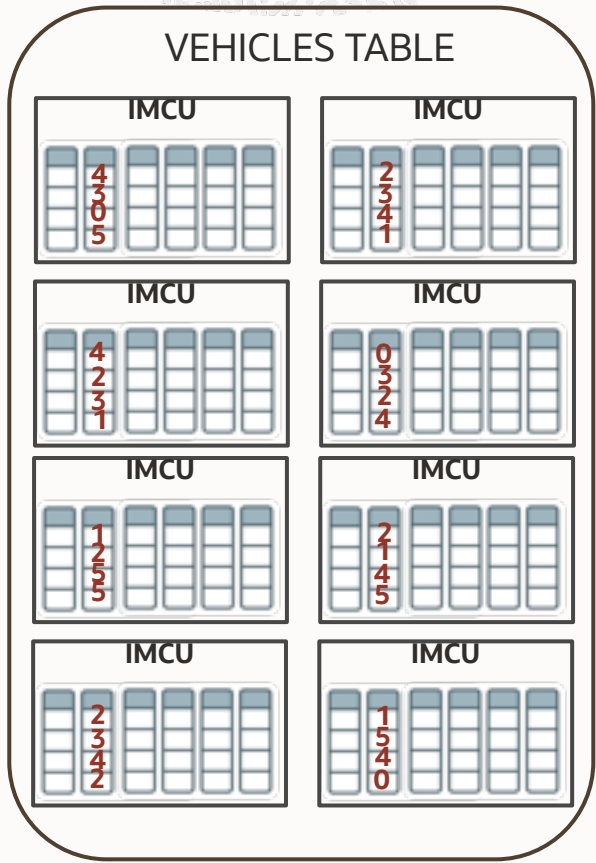
[illegible]

Compression

- Multiple levels of compression
 - FOR DML
 - FOR QUERY LOW/HIGH
 - FOR CAPACITY LOW/HIGH
- Query Low and High use dictionary and run length encoding – evaluated directly against compressed data
- Capacity Low and High add additional “zip-like” compression

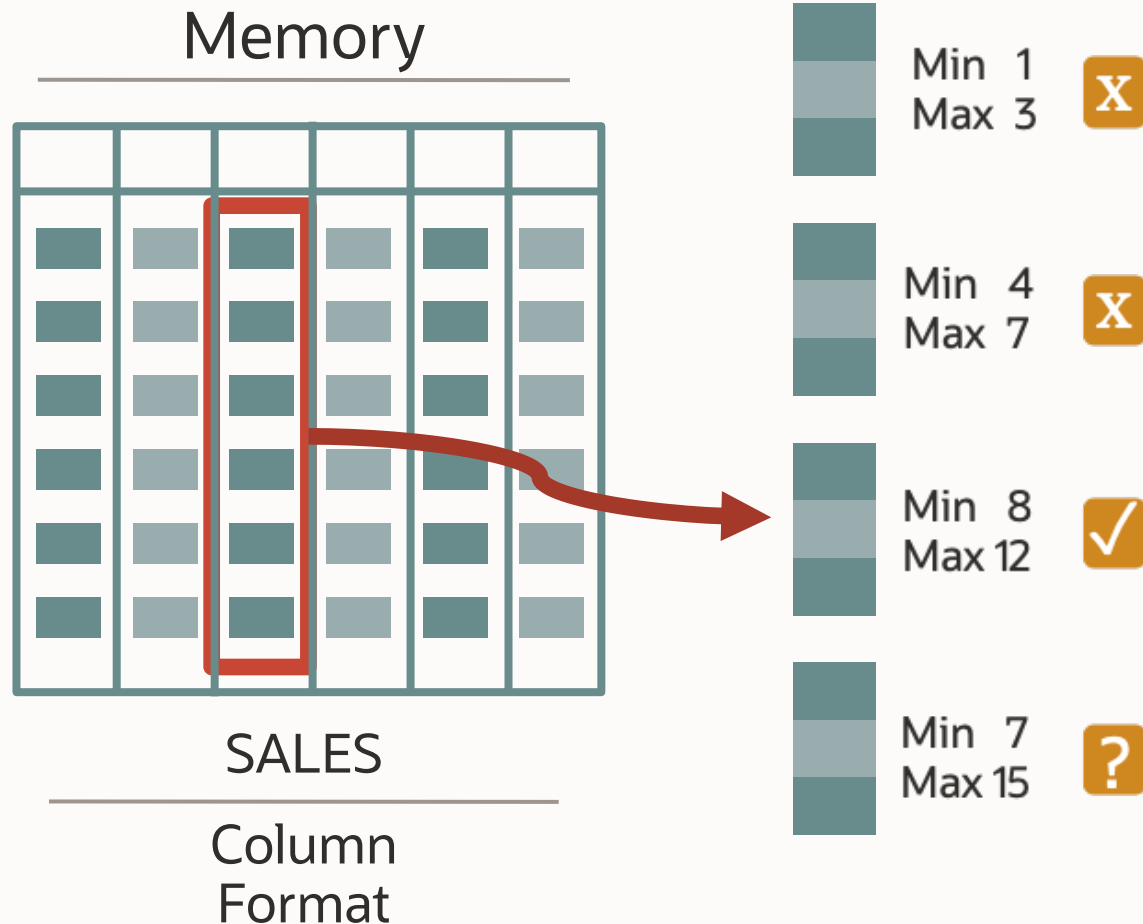
Common Dictionary

NAME	ID
AUDI	0
BMW	1
CADILLAC	2
PORSCHE	3
TESLA	4
VW	5



In-Memory Storage Indexes

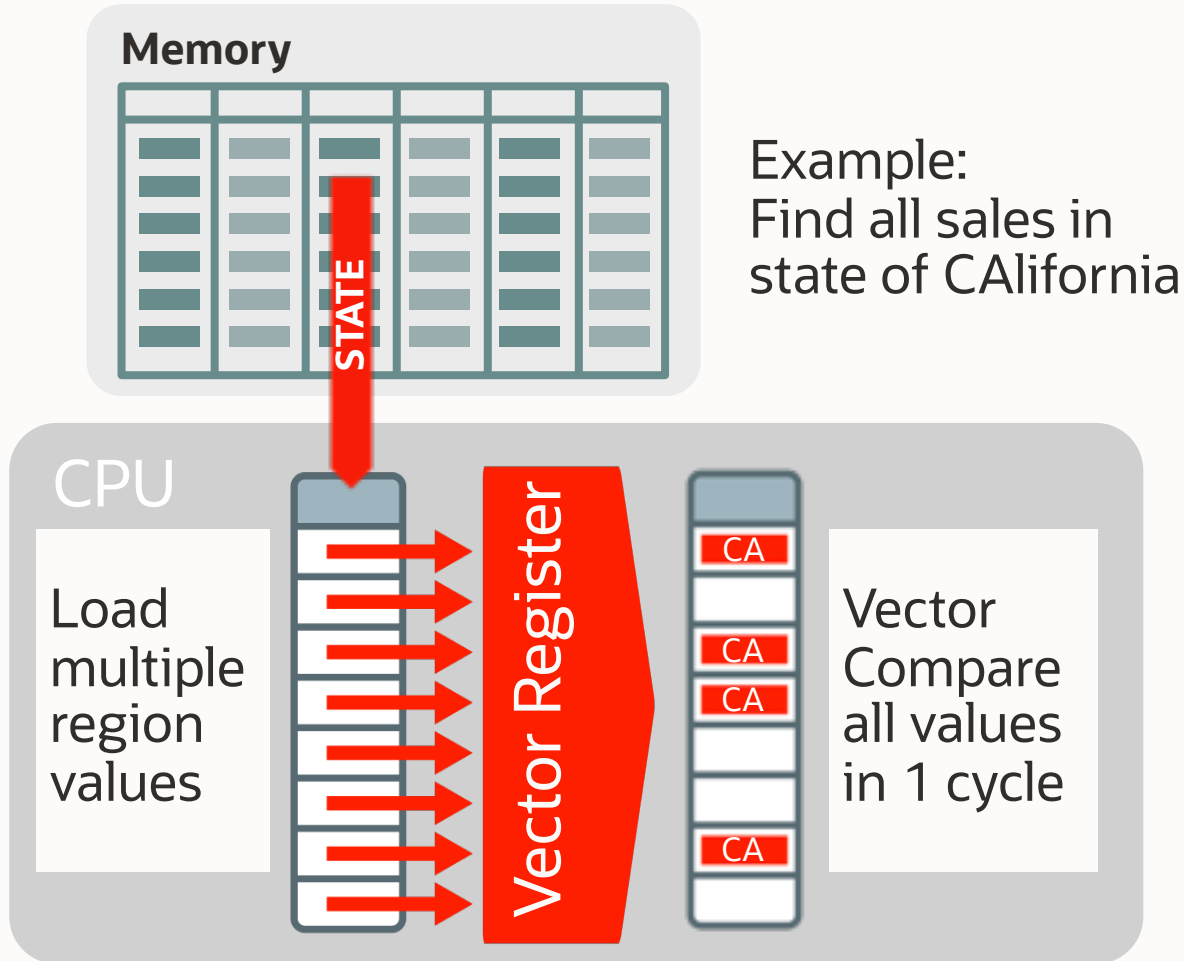
Only look at the data you need!



- Example: Find all sales from stores with a store_id of 8
 - Each column is made up of multiple column units
 - Min / max value is recorded for each column unit in a storage index
 - Storage index provides partition pruning like performance for **ALL** queries

SIMD Vector Processing

Orders of Magnitude Faster Analytic Data Scans



> 10x Faster

- Each CPU core scans local in-memory columns
- SIMD vector instructions used to process multiple values in each instruction
 - Originally designed for graphics & science
- Billions of rows/sec scan rate per CPU core
 - Row format is millions/sec

Optimizer Enhancements



- [illegible]

Identifying Benefit of Pushed Predicates

```
SELECT Count(*) FROM lineorder WHERE lo_partkey = 210876;
```

Count(*)

101

PLAN_TABLE_OUTPUT

Plan hash value: 2267213921

Id	Operation	Name
0	SELECT STATEMENT	
1	SORT AGGREGATE	
* 2	TABLE ACCESS INMEMORY FULL	LINEORDER

Predicate Information (identified by operation id):

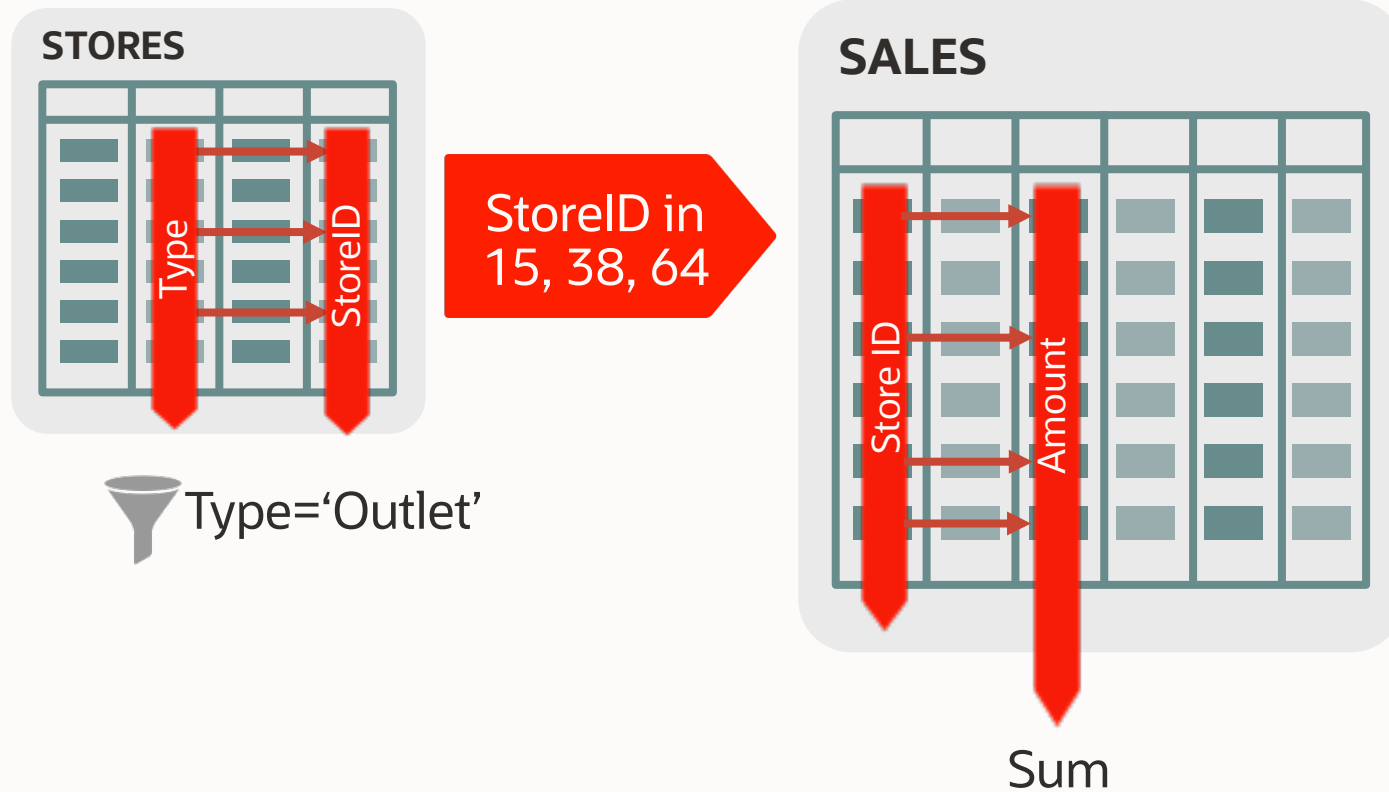
2 - inmemory("LO_PARTKEY"=210876)
filter("LO_PARTKEY"=210876)

NAME	VALUE
IM scan CUs columns accessed	44
IM scan CUs columns theoretical max	748
IM scan CUs memcompress for query low	44
IM scan CUs predicates applied	44
IM scan CUs predicates received	44
IM scan CUs split pieces	63
IM scan bytes in-memory	1196969129
IM scan bytes uncompressed	2285455994
IM scan rows	23996604
IM scan rows projected	101
IM scan rows valid	23996604
IM scan segments minmax eligible	44



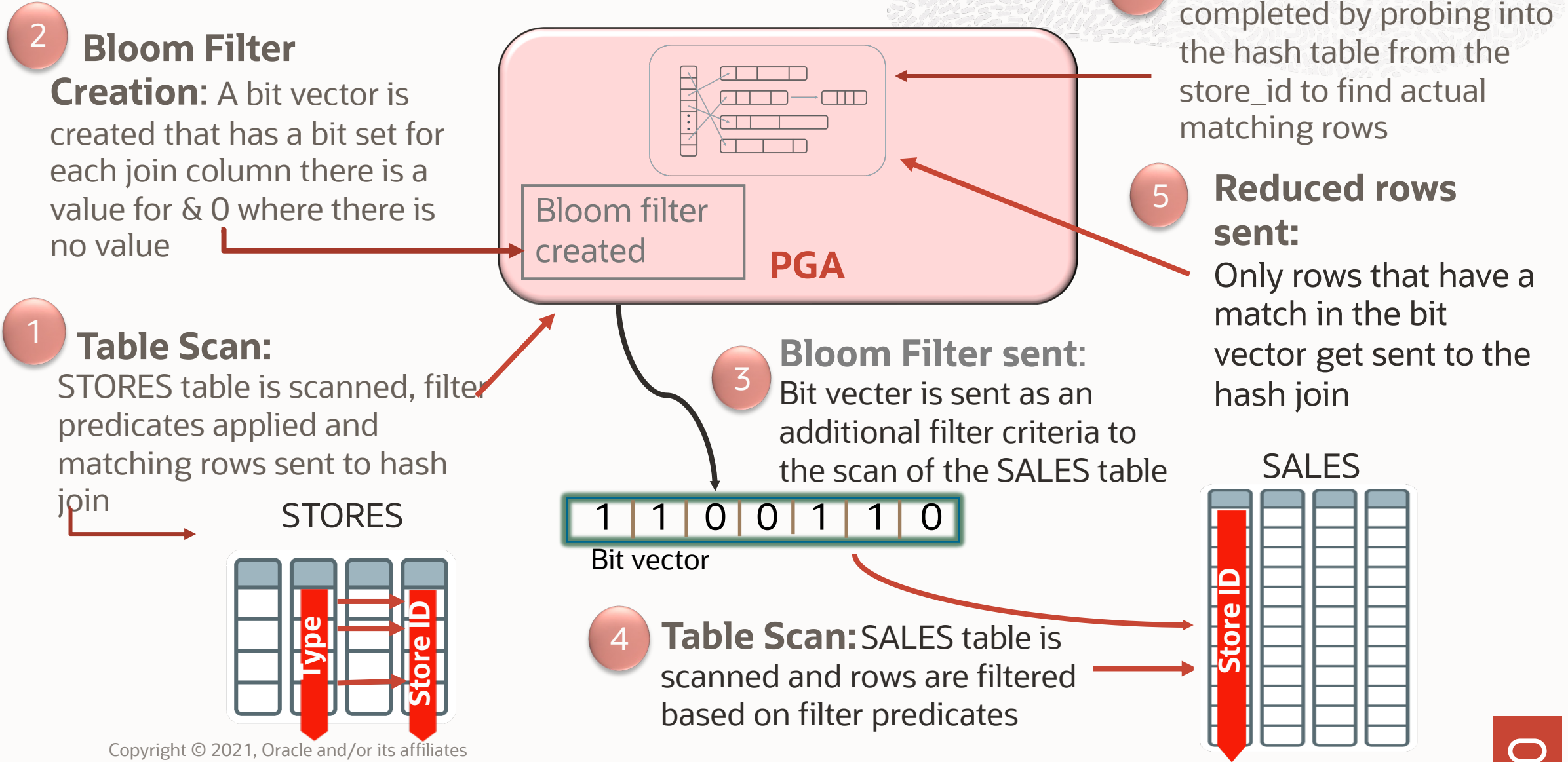
Joining and Combining Data Also Dramatically Faster

Example: Find total sales in outlet stores



- Converts joins of data between two or more tables into fast column scans using Bloom filters
- Joins tables **10x** faster

Database In-Memory Hash Join With Bloom Filters



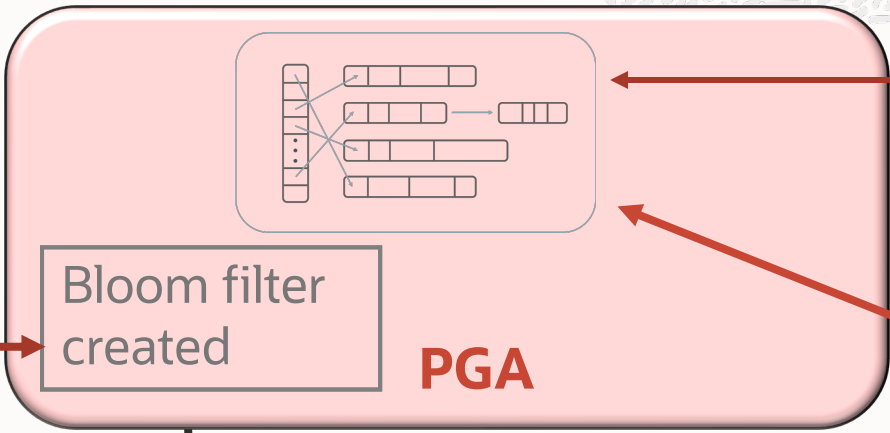
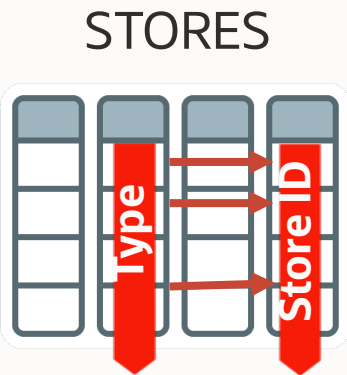
Database In-Memory Hash Join With Vector Join

2 Bloom Filter

Creation: A bit vector is created that has a bit set for each join column there is a value for & 0 where there is no value

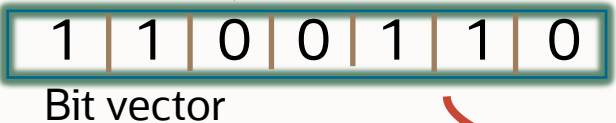
1 Table Scan:

STORES table is scanned, filter predicates applied and matching rows sent to hash join



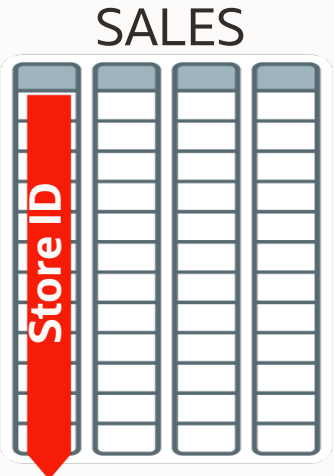
3 Bloom Filter sent:

Bit vector is sent as an additional filter criteria to the scan of the SALES table



4 Table Scan:

SALES table is scanned and rows are filtered based on filter predicates

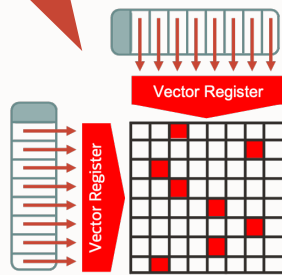


6 Hash Join:

Join completed by probing into the hash table from the store_id to find actual

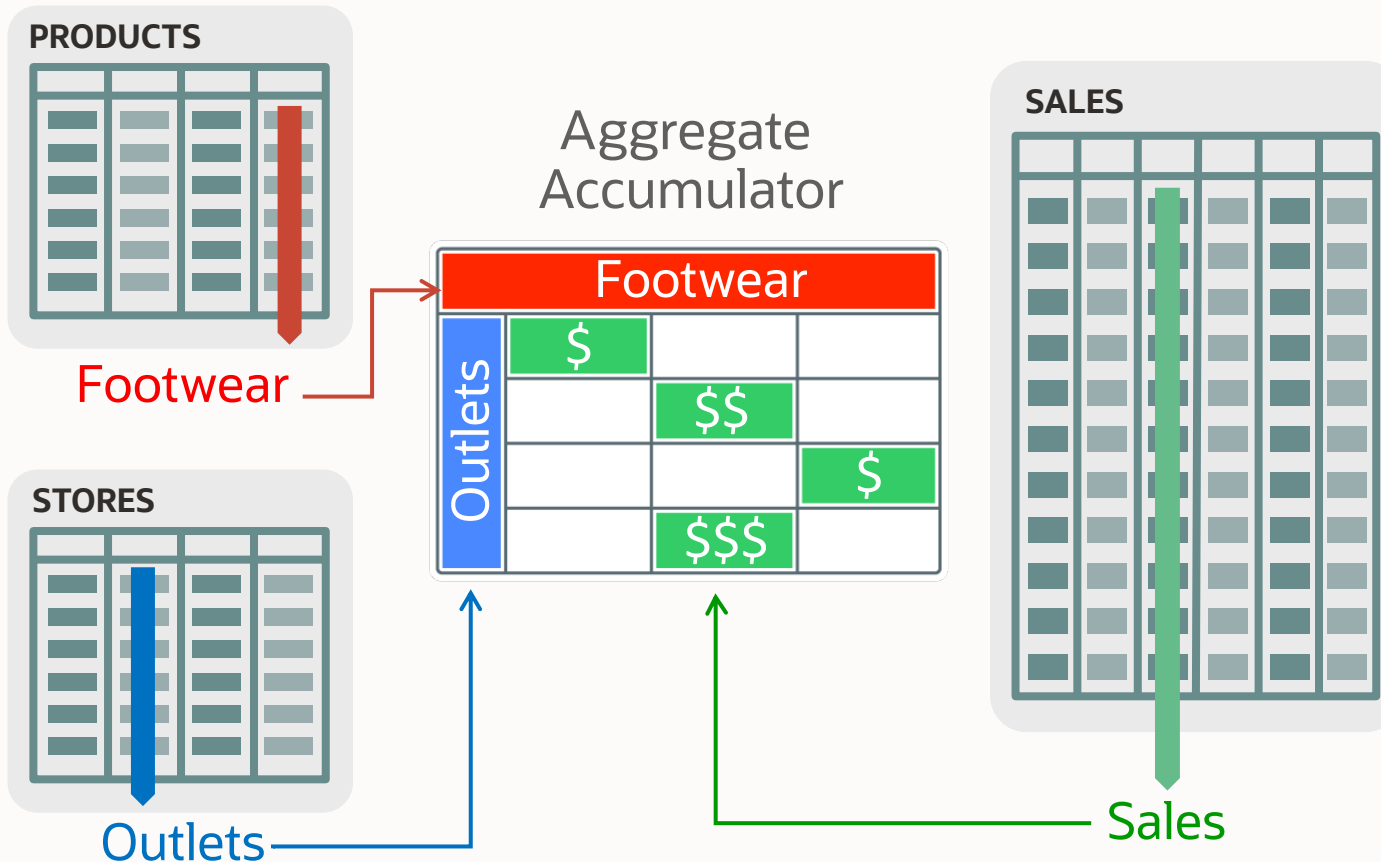
New in 21c: In-Memory Vector Join

match in a vector get sent to the hash join



In-Memory Aggregation

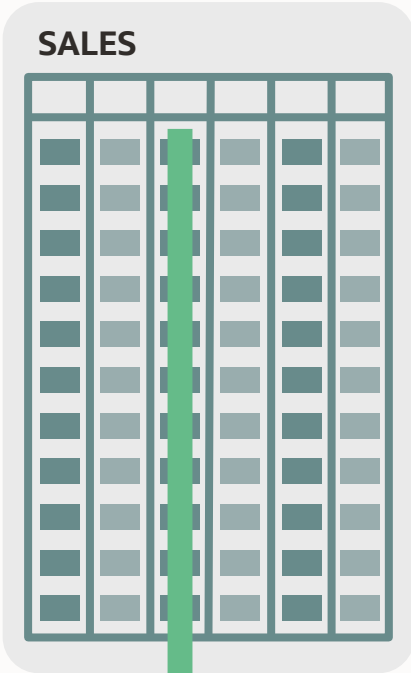
Example: Report sales of footwear in outlet stores



- Execution plan shows **Vector Group By**
- Dynamically creates in-memory report outline (aggregate accumulator)
- Aggregation performed in-memory during fast fact scan
- Key vectors are used instead of Bloom filters
- Key vectors use dense grouping keys to map all key combinations

Key Vector Use & Vector Group By

Id	Operation	Name
0	SELECT STATEMENT	
1	TEMP TABLE TRANSFORMATION	
2	LOAD AS SELECT(CURSOR DURATION MEMORY)	SYS_TEMP_0FD9DADAD_9873DD
3	VECTOR GROUP BY	
4	KEY VECTOR CREATE BUFFERED	:KV0000
5	PARTITION RANGE ALL	
6	TABLE ACCESS INMEMORY FULL	PRODUCTS
7	LOAD AS SELECT(CURSOR DURATION MEMORY)	SYS_TEMP_0FD9DADAE_9873DD
8	VECTOR GROUP BY	
9	KEY VECTOR CREATE BUFFERED	:KV0001
10	TABLE ACCESS INMEMORY FULL	STORES
11	HASH GROUP BY	
12	HASH JOIN	
13	HASH JOIN	
14	TABLE ACCESS FULL	SYS_TEMP_0FD9DADAE_9873DD
15	VIEW	VW_VT_AF278325
16	VECTOR GROUP BY	
17	HASH GROUP BY	
18	KEY VECTOR USE	:KV0001
19	KEY VECTOR USE	:KV0000
20	PARTITION RANGE SUBQUERY	
21	TABLE ACCESS INMEMORY FULL	SALES
22	TABLE ACCESS FULL	SYS_TEMP_0FD9DADAD_9873DD



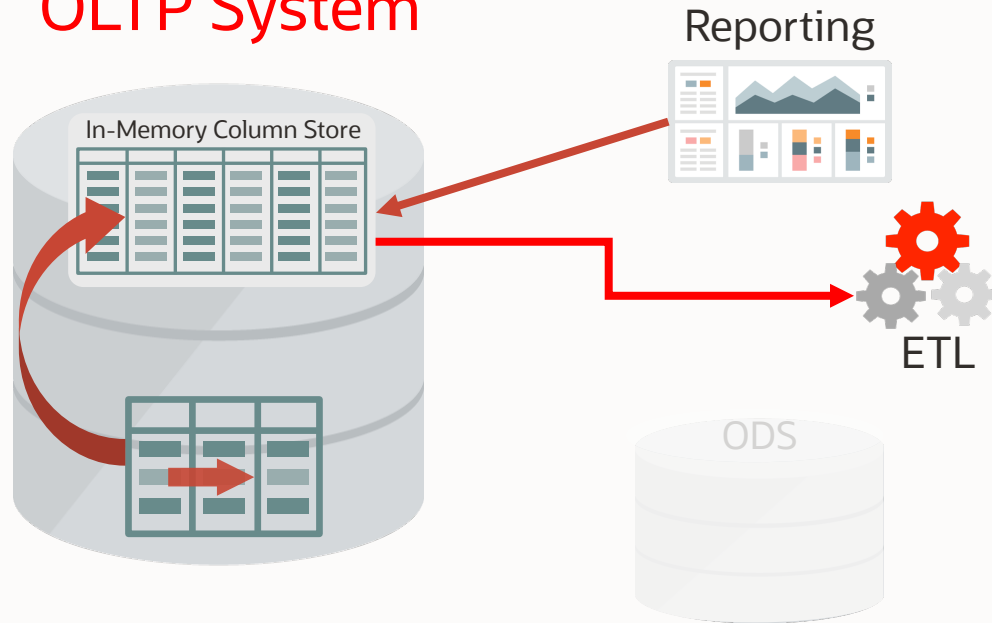
Scan, filter and aggregate



When and Where Should I Use Database In-Memory?

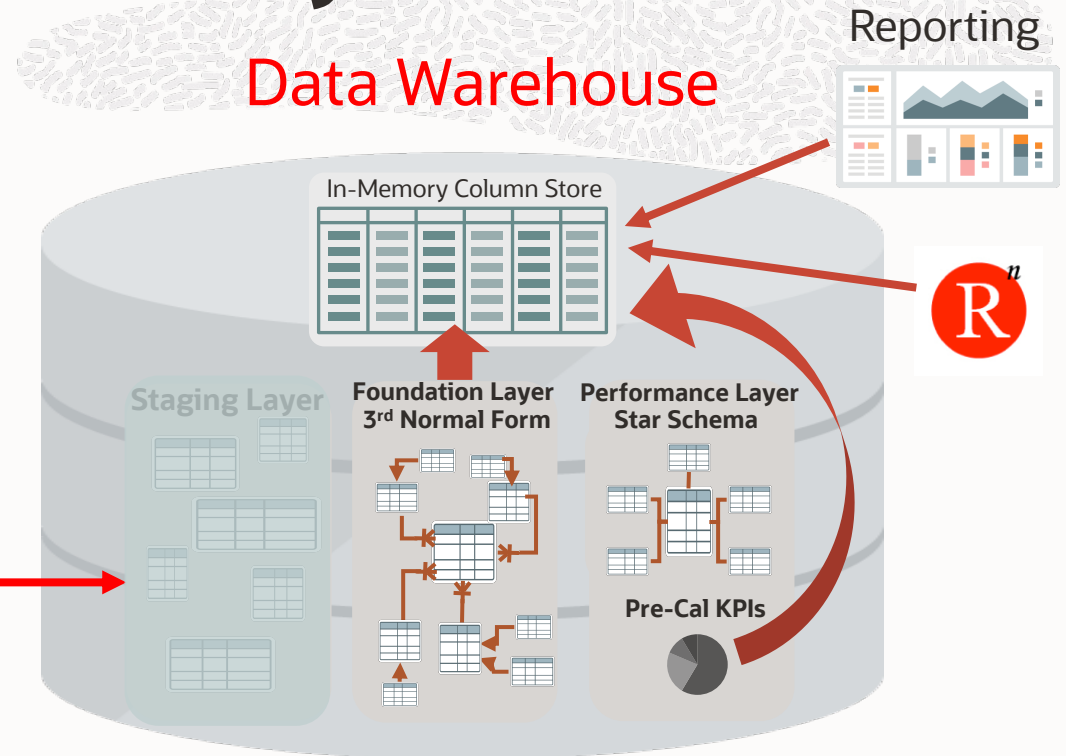
Where to Use Database In-Memory

OLTP System



- Enables real-time reporting directly on OLTP data
- Speeds data extraction part of ETL process
- Removes need for separate ODS
- Speeds up mixed workload

Data Warehouse



- Star-schema and pre-calculated KPIs
 - Improves performance of dash-boards
- All or a subset of Foundation Layer
 - For time-sensitive analytics on 3rd normal form
- Staging/ETL/Temp not good candidates
 - Write once, read once

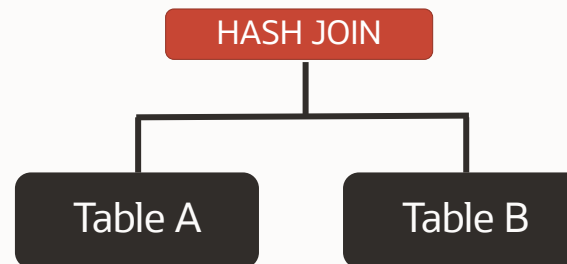
Which Queries Benefit From Database In-Memory?

For a non-trivial amount of rows and execution time, when a significant amount of time ...

is spent accessing data



is spent joining data



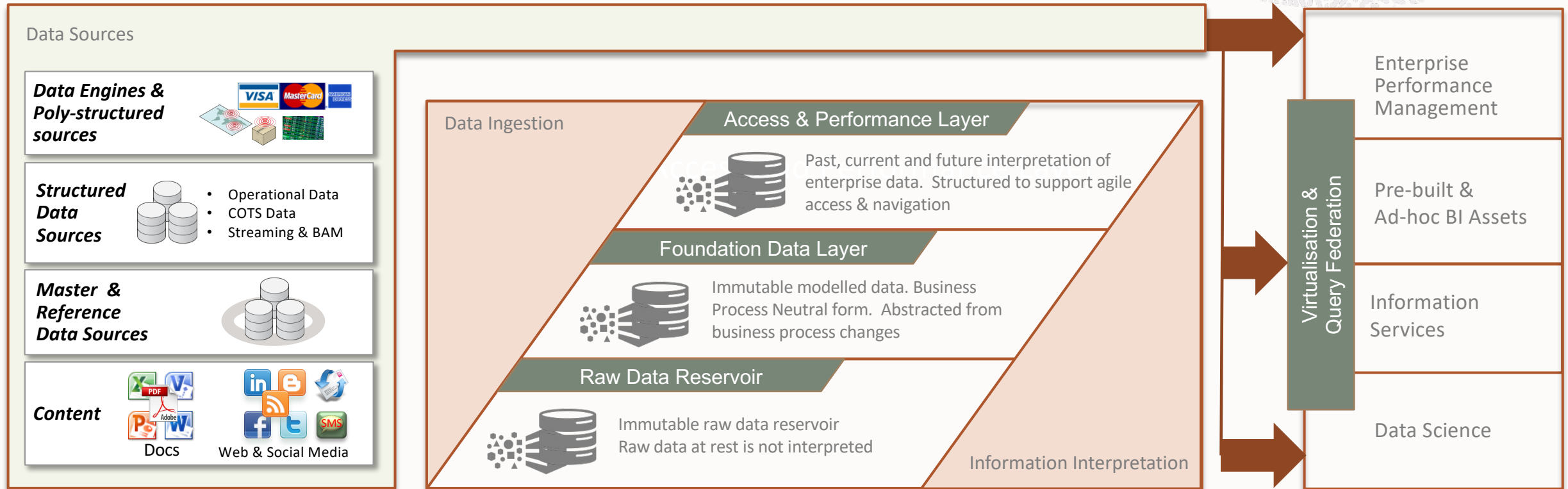
is spent aggregating data



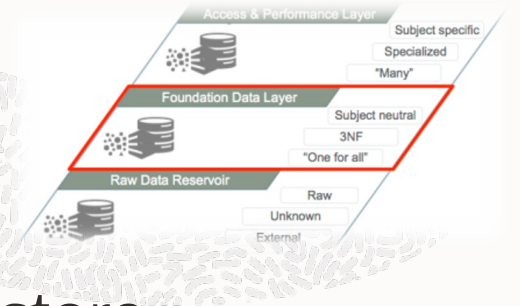
High Level Use Cases

Data Warehouse Systems

Oracle Information Management Architecture



In-Memory for Foundation Data Layer



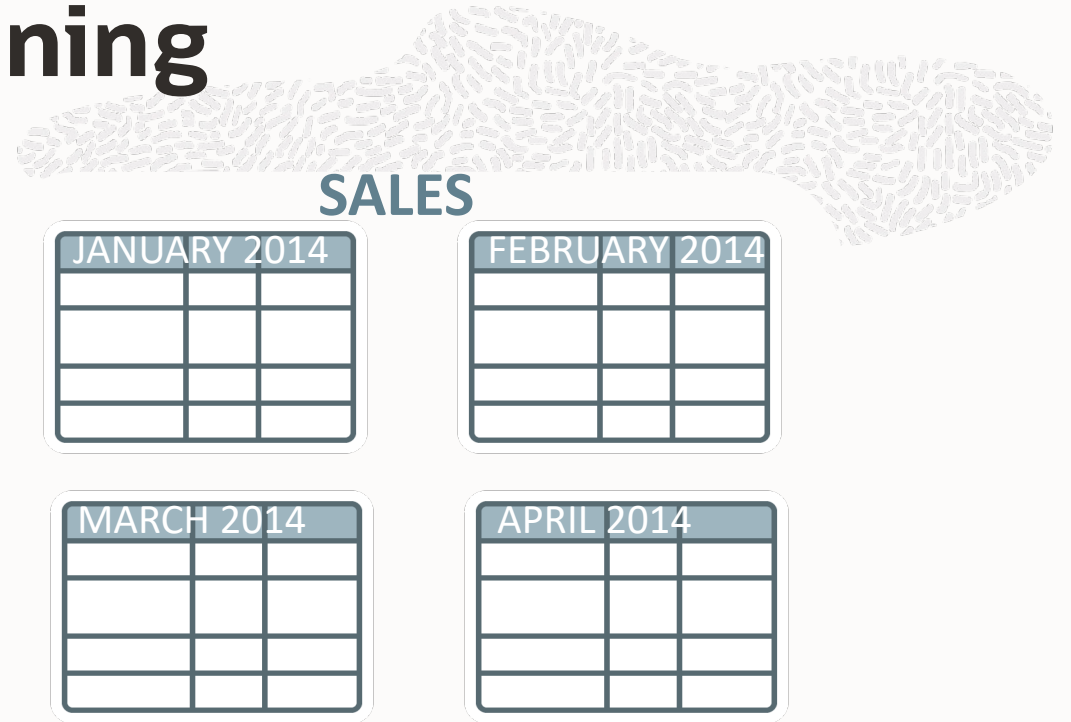
- Foundation layer is a good candidate for In-Memory column store
- Foundation layer contains a full copy of the transactional data
- By partitioning the data only the performance sensitive data is populated in the IM column store
- Parallel execution enables near linear scalability of In-Memory scans
- Joins are executed more efficiently using Bloom filters

In-Memory's FFL* - Partitioning



Large Table

Difficult to Manage



Partitions

Divide and Conquer

Easier to Manage

Improve Performance

Transparent to applications

Populating partitions into the In-Memory Column Store

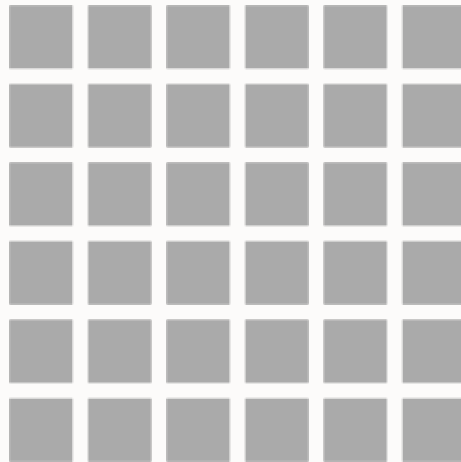
```
CREATE TABLE ORDERS .....  
PARTITION BY RANGE .....  
  (PARTITION p1 .....  
    INMEMORY MEMCOMPRESS FOR DML ,  
   PARTITION p2 .....  
    INMEMORY MEMCOMPRESS FOR QUERY ,  
   PARTITION p3 .....  
    INMEMORY MEMCOMPRESS FOR CAPACITY  
   :  
   PARTITION p200 .....  
   NO INMEMORY  
  ) ;
```

- Not all partitions need to be populated in-memory
- Different levels of MEMCOMPRESS allow more data to be populated
- Easy to switch levels as part of ILM strategy
- Smart processing through table expansion for partially populated partitioned tables

What Are Traditional Summary Objects?



INDEXES

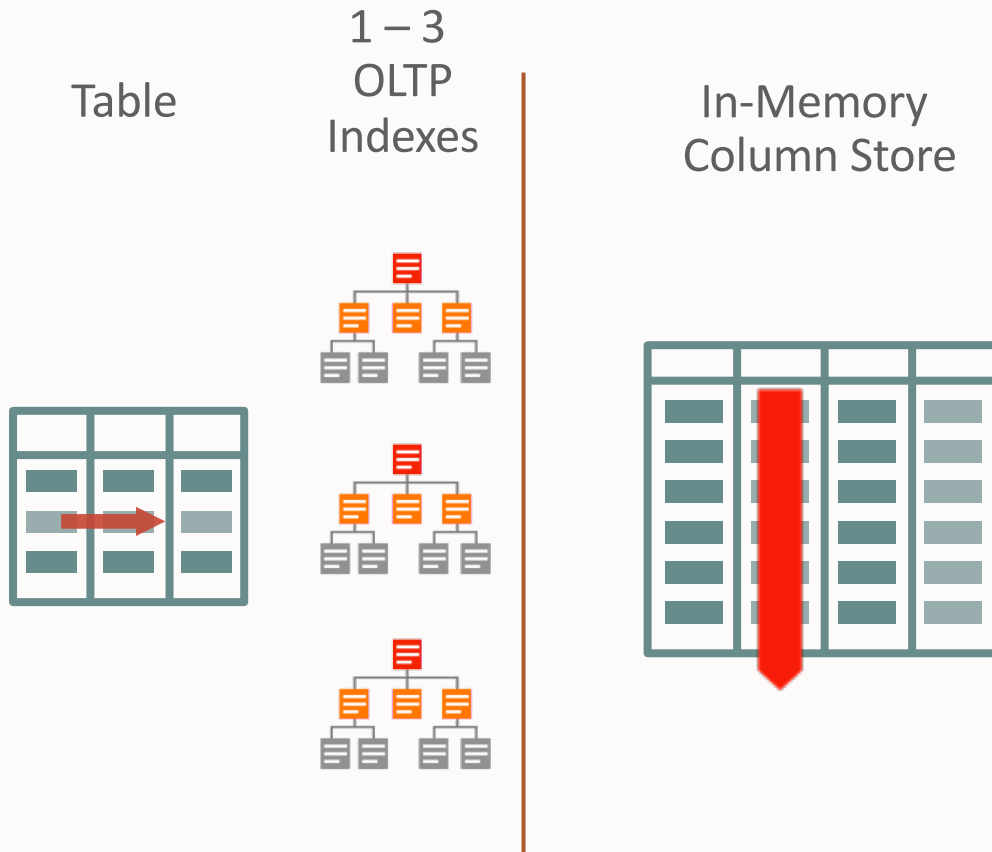


OLAP CUBES



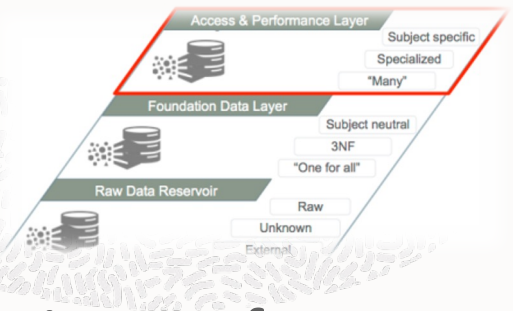
MATERIALIZED
VIEWS

In-Memory Column Store Replaces Analytic Indexes

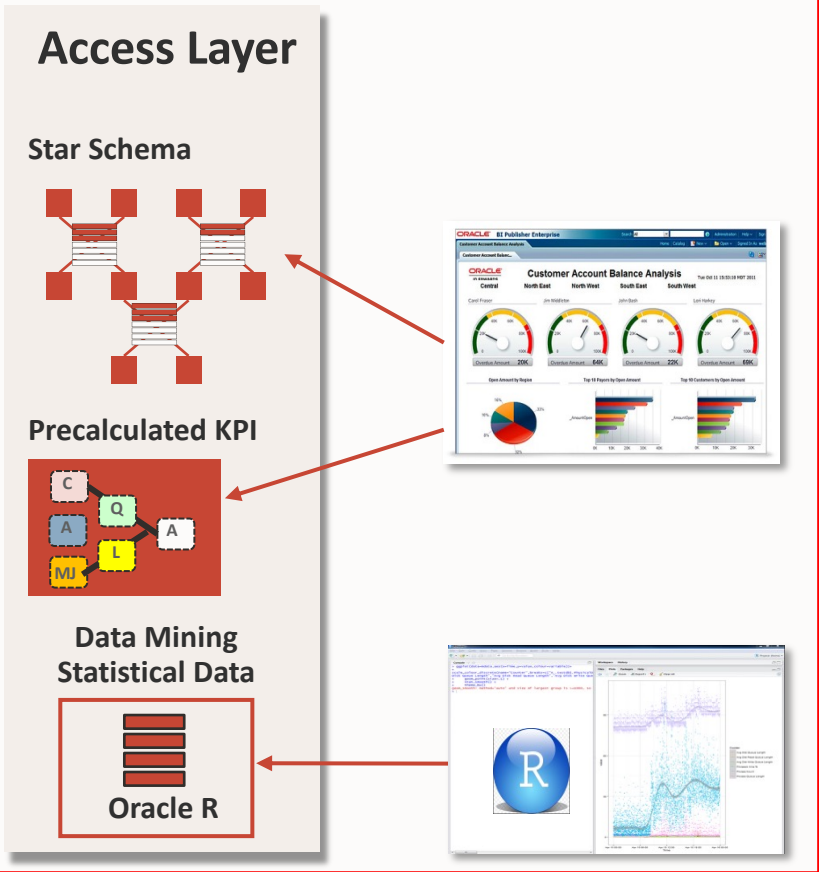


- Fast analytics on any columns
 - Better for unpredictable analytics
 - Less tuning & administration
- Column Store not persistent so update cost is much lower
 - OLTP & batch run faster

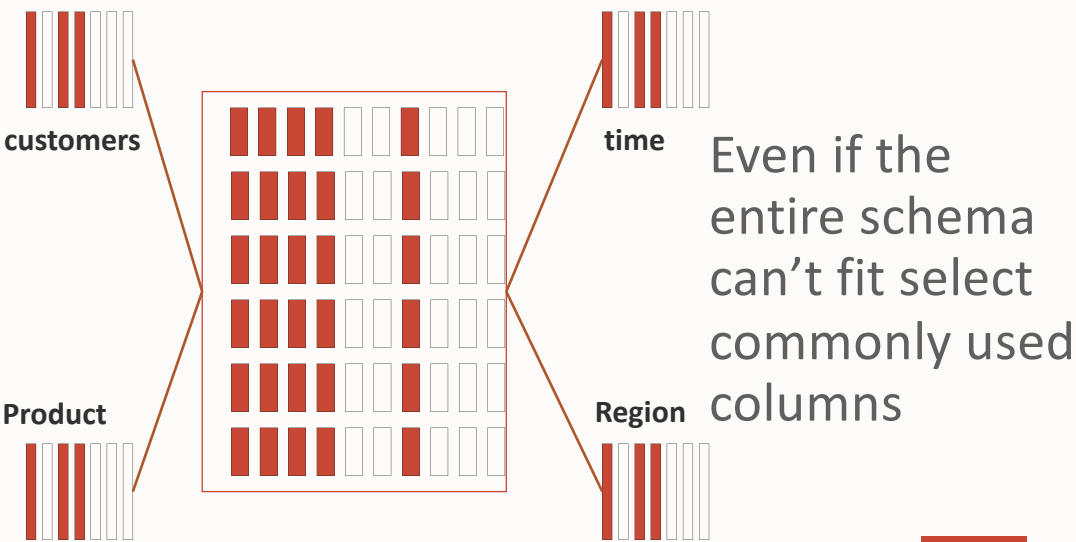
In-Memory and Data Warehouse Star Schema



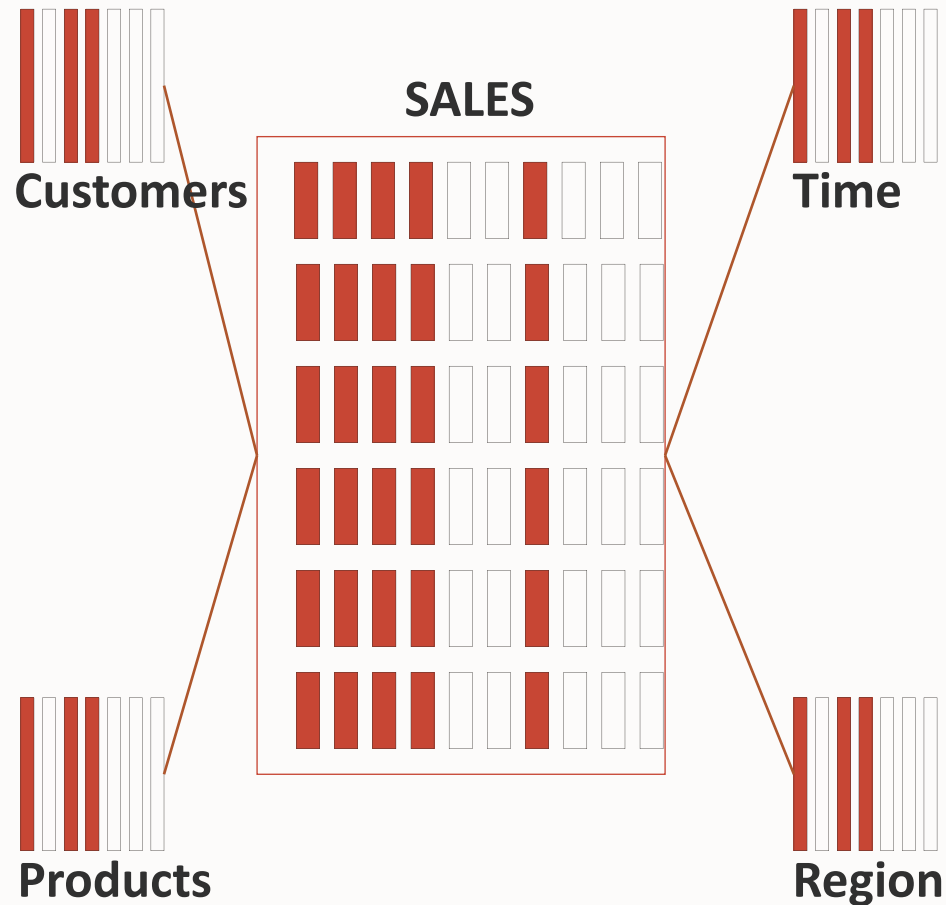
Enterprise Data Warehouse



- Star-Schema makes it easy for end-users to navigate data
 - Illustrates business context
- In-Memory Aggregation makes querying star-schemas very fast



In-Memory and Star Schema



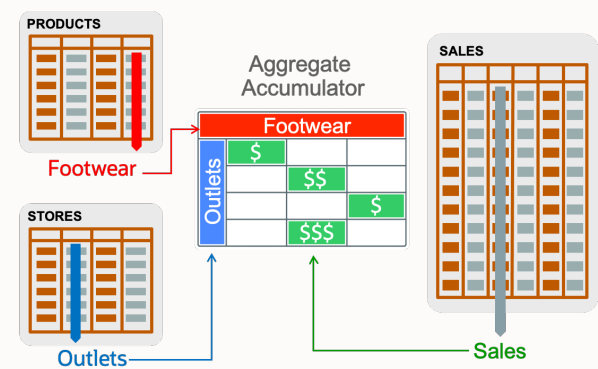
- If not enough memory to accommodate entire schema populate only commonly used columns

```
ALTER TABLE sales INMEMORY  
NO INMEMORY (TAX_RATE, COMMISSON);
```

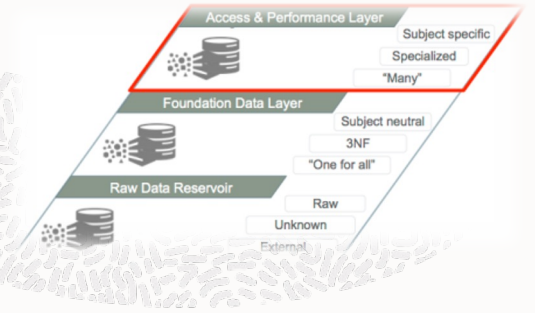
In-Memory Aggregation : Vector Group By

* 23	HASH JOIN	
* 24	HASH JOIN	
25	TABLE ACCESS FULL	SYS_TEMP_0FD9D662E_2D4940
26	VIEW	VW_VT_80F21617
27	HASH GROUP BY	
28	PX RECEIVE	
29	PX SEND HASH	:TQ50000
30	VECTOR GROUP BY	
31	HASH GROUP BY	
32	KEY VECTOR USE	:KV0000
33	KEY VECTOR USE	:KV0001
34	PX BLOCK ITERATOR	
* 35	TABLE ACCESS INMEMORY FULL	SALES
36	TABLE ACCESS FULL	SYS_TEMP_0FD9D662F_2D4940
37	TABLE ACCESS FULL	SYS_TEMP_0FD9D662D_2D4940

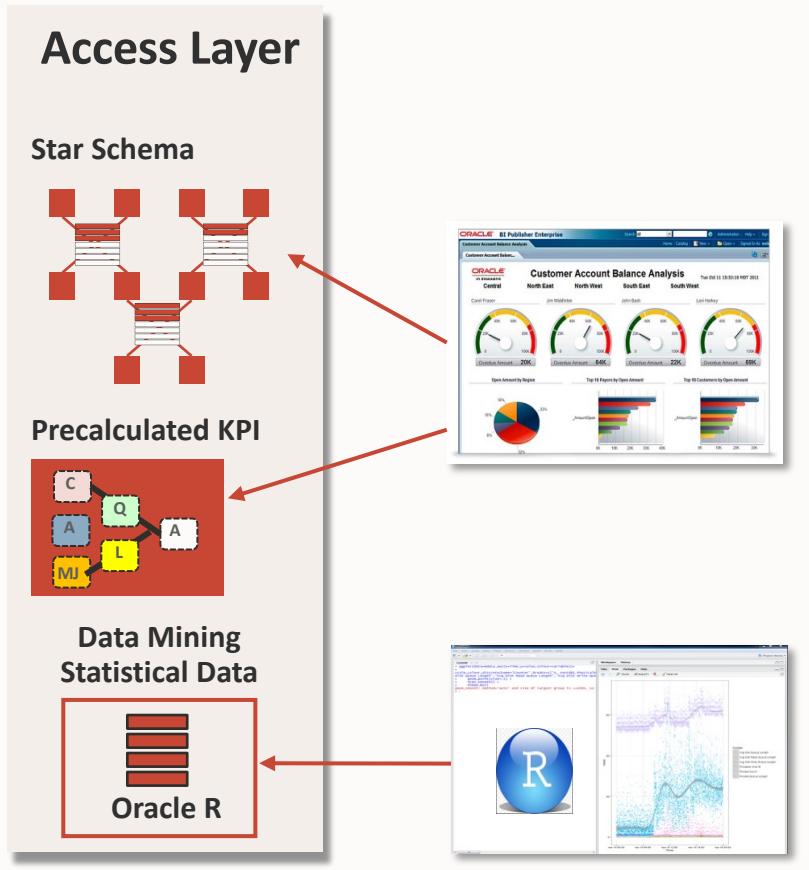
Scan the SALES table with Vector Group By



In-Memory and Data Warehouse Pre-calculated KPIs



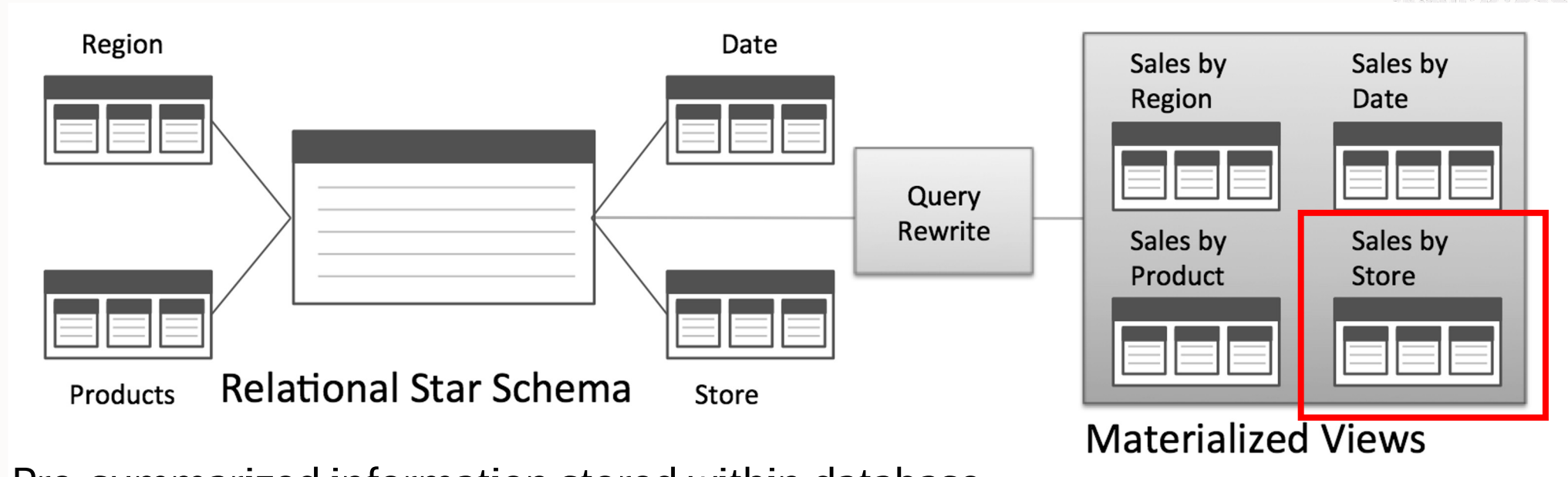
Enterprise Data Warehouse



- Up to 80% of all BI analysis are predictable and should be pre-calculated
- Persist pre-calculated KPIs via materialized views
- Populate materialized view in IM column store to speed up end-user dashboards
- Enable query rewrite so similar SQL can access the same MVs

Summary Management

Improve response time with Materialized Views



- Pre-summarized information stored within database
 - Pre-calculated KPIs
- Supports sophisticated transparent query rewrite
- Fast incremental refresh of changed data



What are the
total sales figures
by store

Materialized Views



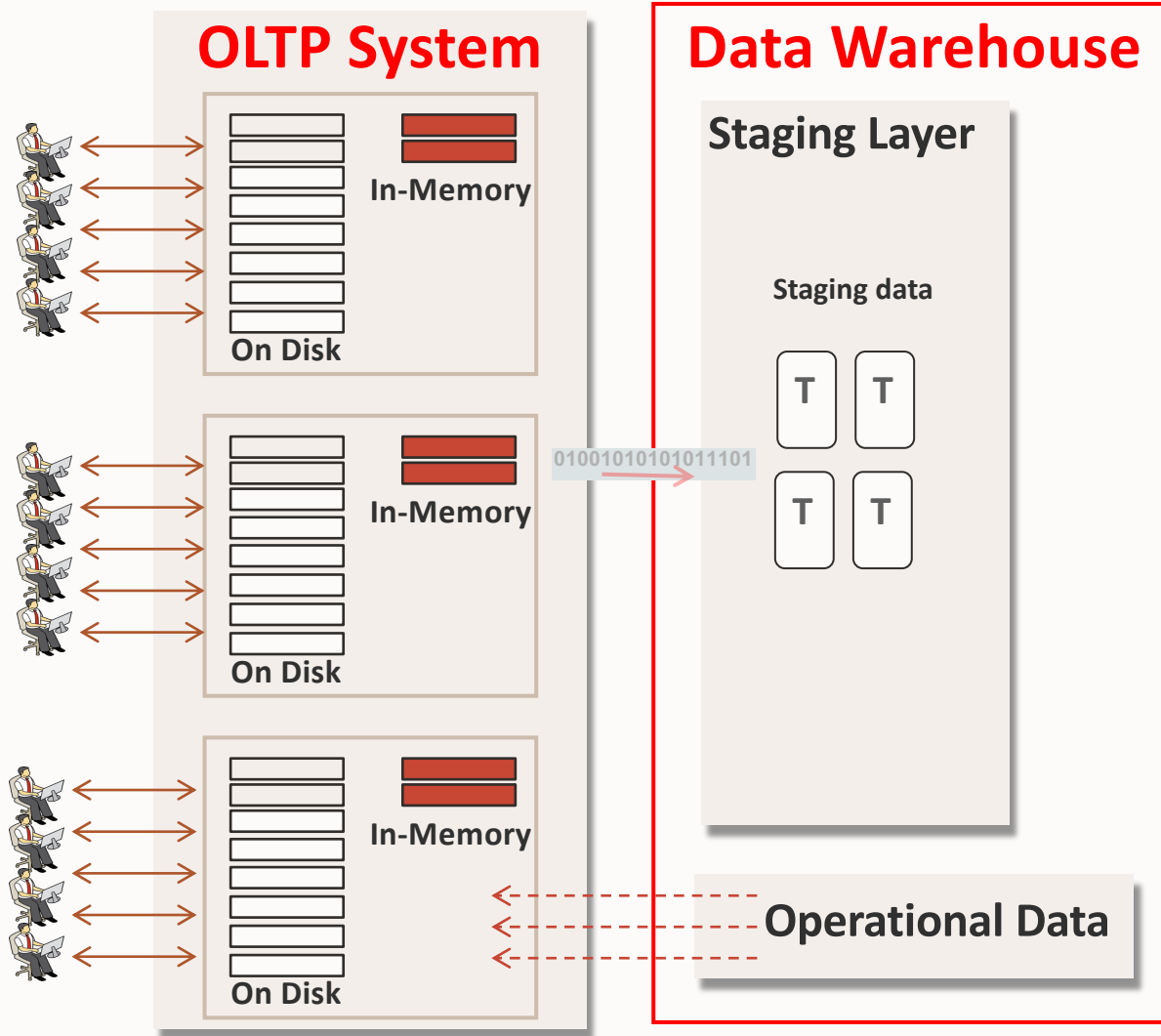
- Summary object containing pre-calculated results
- Typically created to speed up resource intensive joins and aggregations
- Automatically or manually refreshed
- Queries automatically rewritten to use MV where applicable
- Underlying table can be partitioned
- Multiple rewrite capabilities (i.e. text match, general, etc.)

**Excellent candidate to be put in the In-Memory Column Store
Together they speed up all aspects of the execution plan**

High Level Use Cases

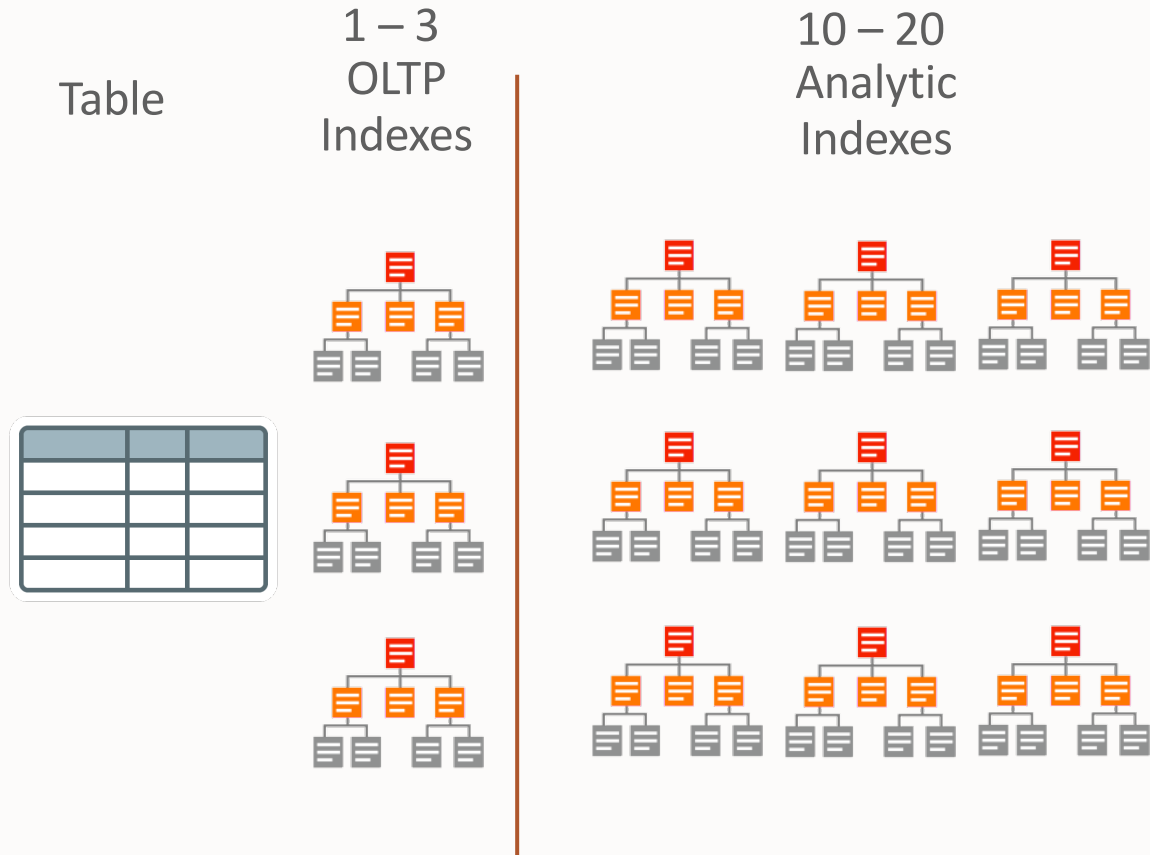
Enterprise OLTP Systems

In-Memory and OLTP



- Enables real-time reporting directly on OLTP source data
- In-memory transactional data can be used for ETL while classic row-store is used for transactions
- May remove the need for ODS

Complex OLTP is Slowed by Analytic Indexes



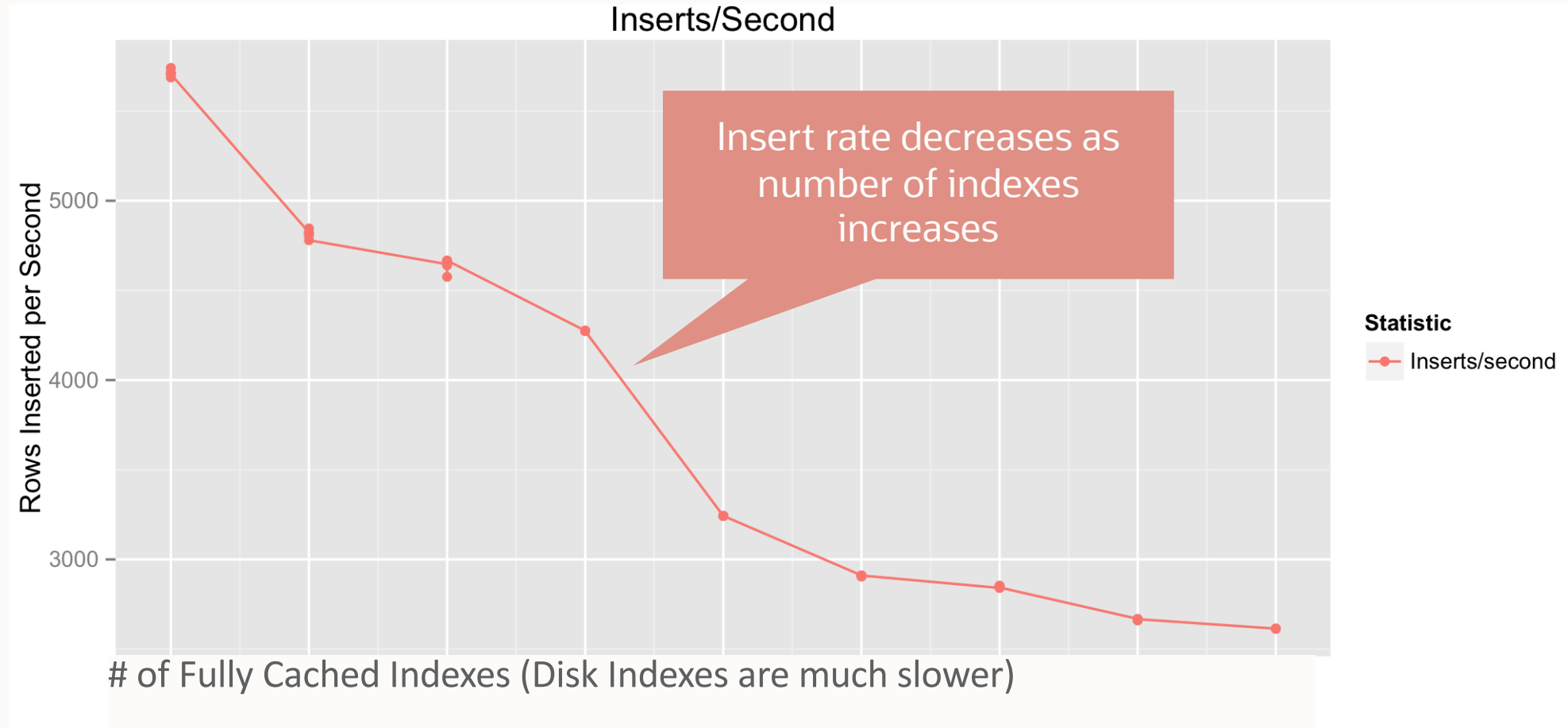
Most Indexes in complex OLTP (e.g. ERP) databases are only used for analytic queries

Inserting one row into a table requires updating 10-20 analytic indexes:
Slow!

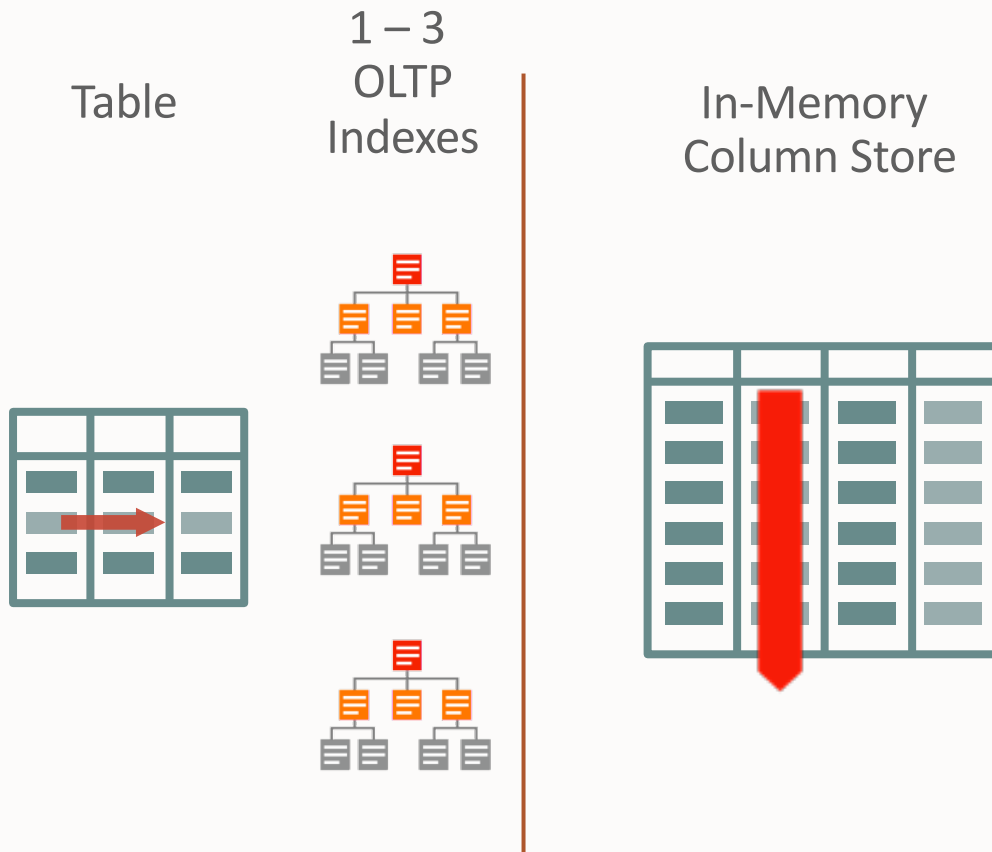
Indexes only speed up predictable queries & reports

Can Also Speed Up DML

OLTP is Slowed Down by Analytic Indexes



In-Memory Column Store Replaces Analytic Indexes



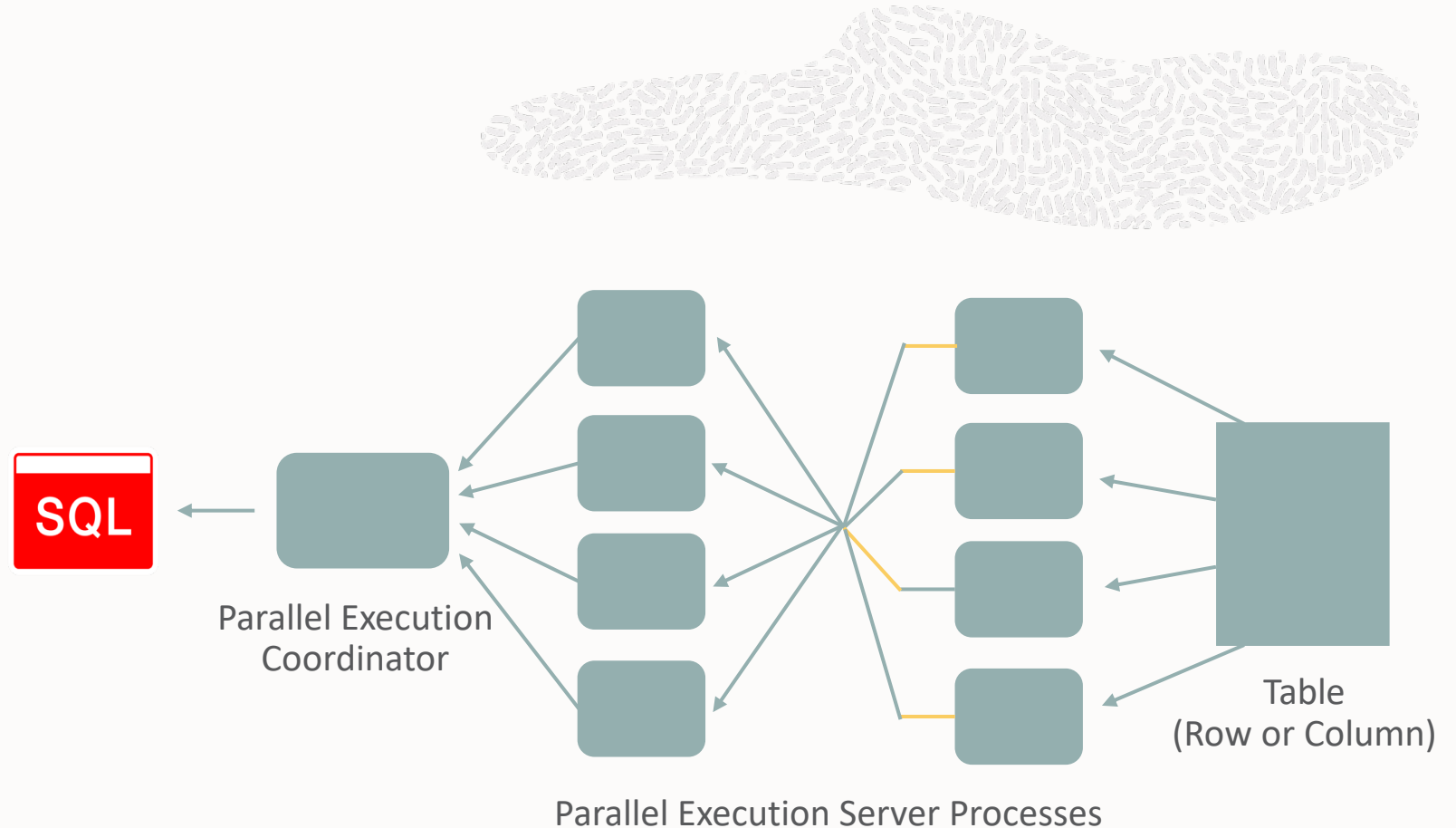
- Fast analytics on any columns
 - Better for unpredictable analytics
 - Less tuning & administration
- Column Store not persistent so update cost is much lower
 - OLTP & batch run faster

How do you determine which indexes to drop?

What About Parallelization?

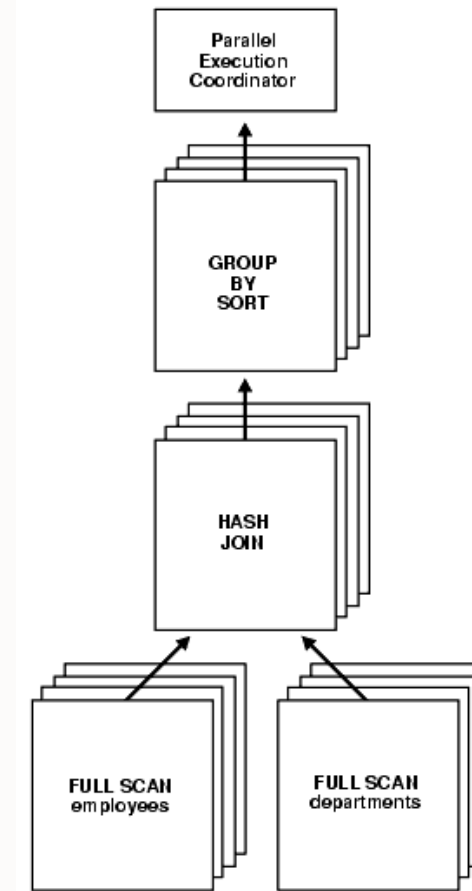
Parallel Query

- Goal: Reduce scan time
- Parallel Query scans an object using multiple processes
- Controlled by the Degree of Parallelism (DOP)
- First introduced in Oracle Release 7.1



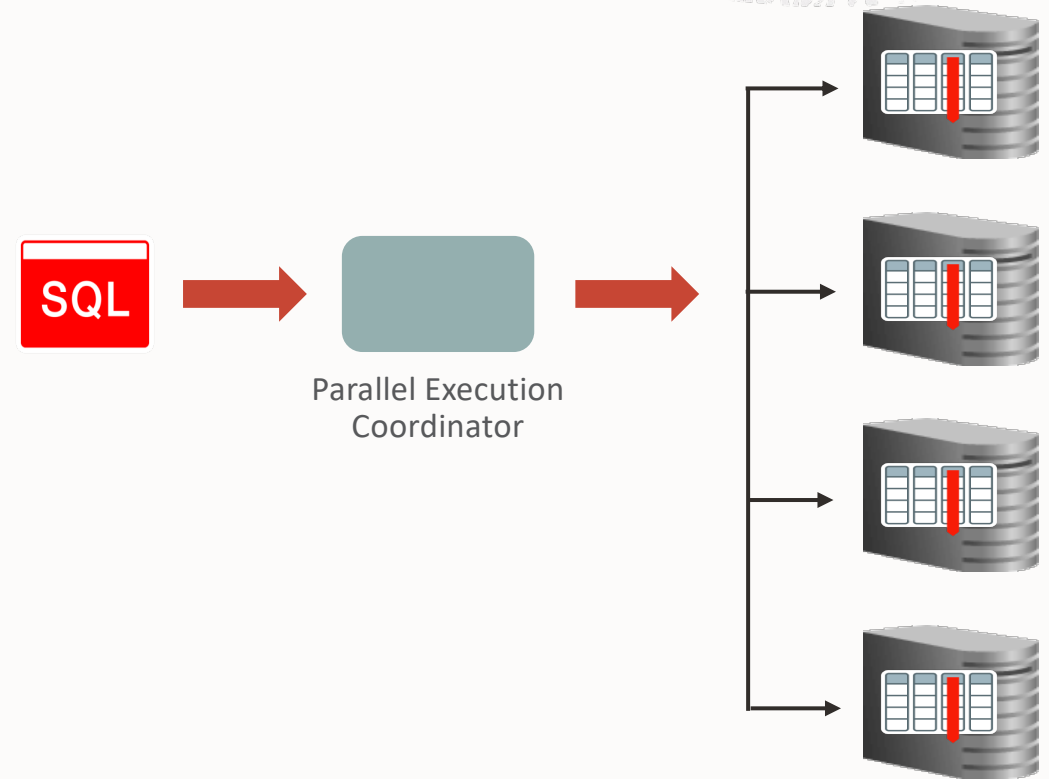
Parallel Query Operations

- Parallel Query is more than just parallelizing access methods
- Parallel Query can parallelize:
 - Join methods
 - Miscellaneous SQL operations
 - Other statements (i.e. DDL, DML)



Parallel Query on RAC with Database In-Memory

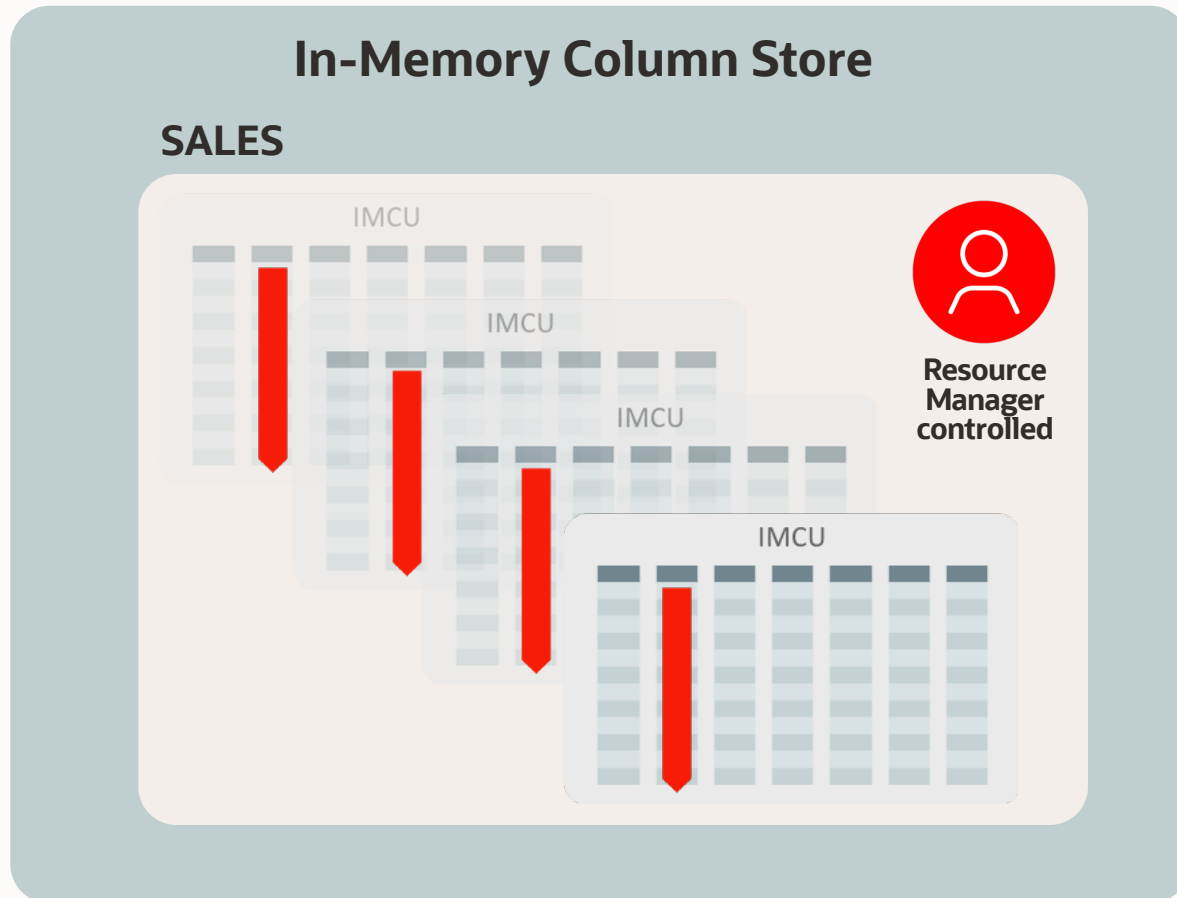
- Scale-Out across servers to grow memory and CPUs
- Shared nothing architecture
- **IMCUs not shipped across interconnect** – cache fusion is not in play!
- In-Memory **queries are parallelized** across servers to access local column data



In-Memory Dynamic Scans

Better Columnar Scan Performance

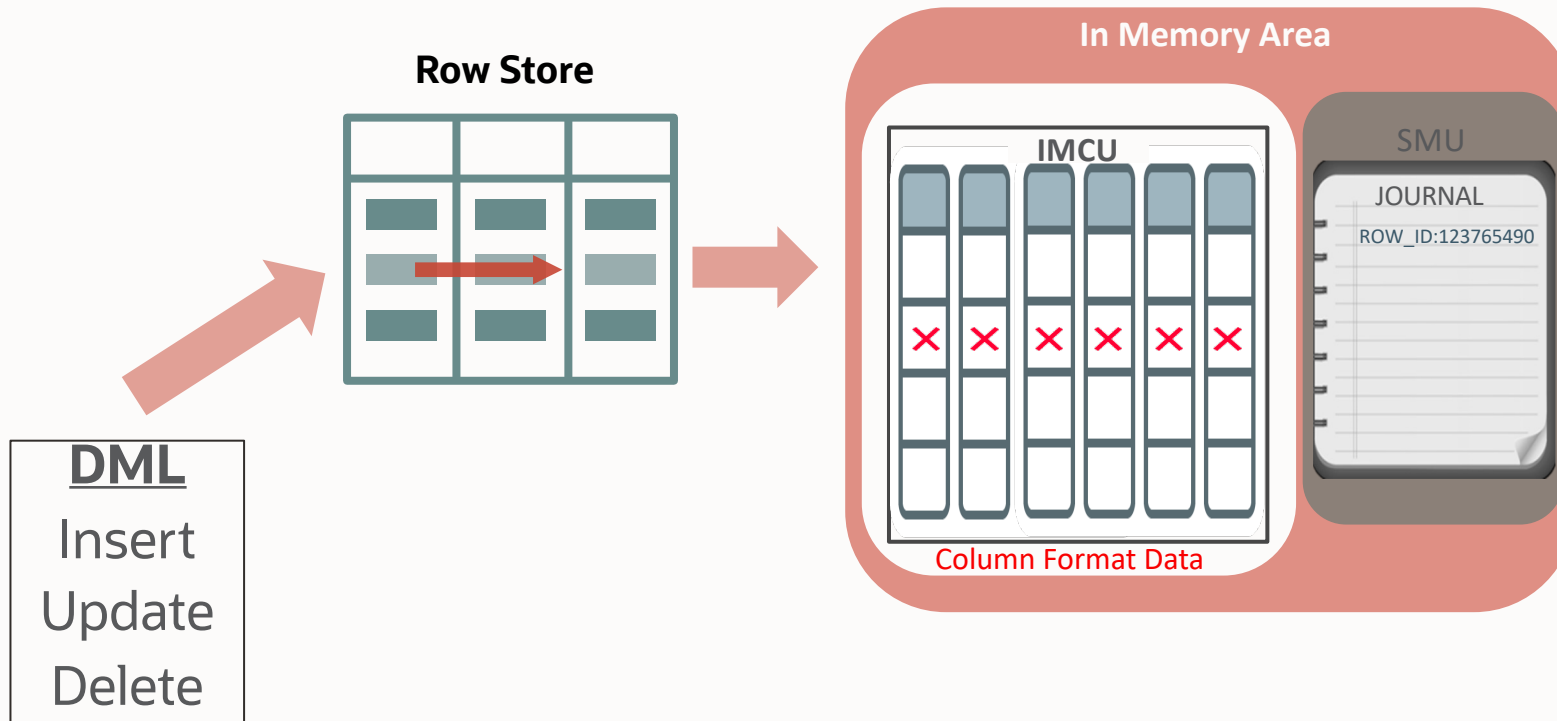
```
SELECT SUM(total) FROM sales
WHERE region = 'CA';
```



- Fast parallelization of simple scans using light-weight threads
- **Scan Multiple** IMCUs at a time
 - **Multiple** threads per scan process
- Number of active threads **dynamically** controlled by Resource Manager
- Fully leverage CPU resources to maximize columnar scan performance
- **Up to 2X performance gains**

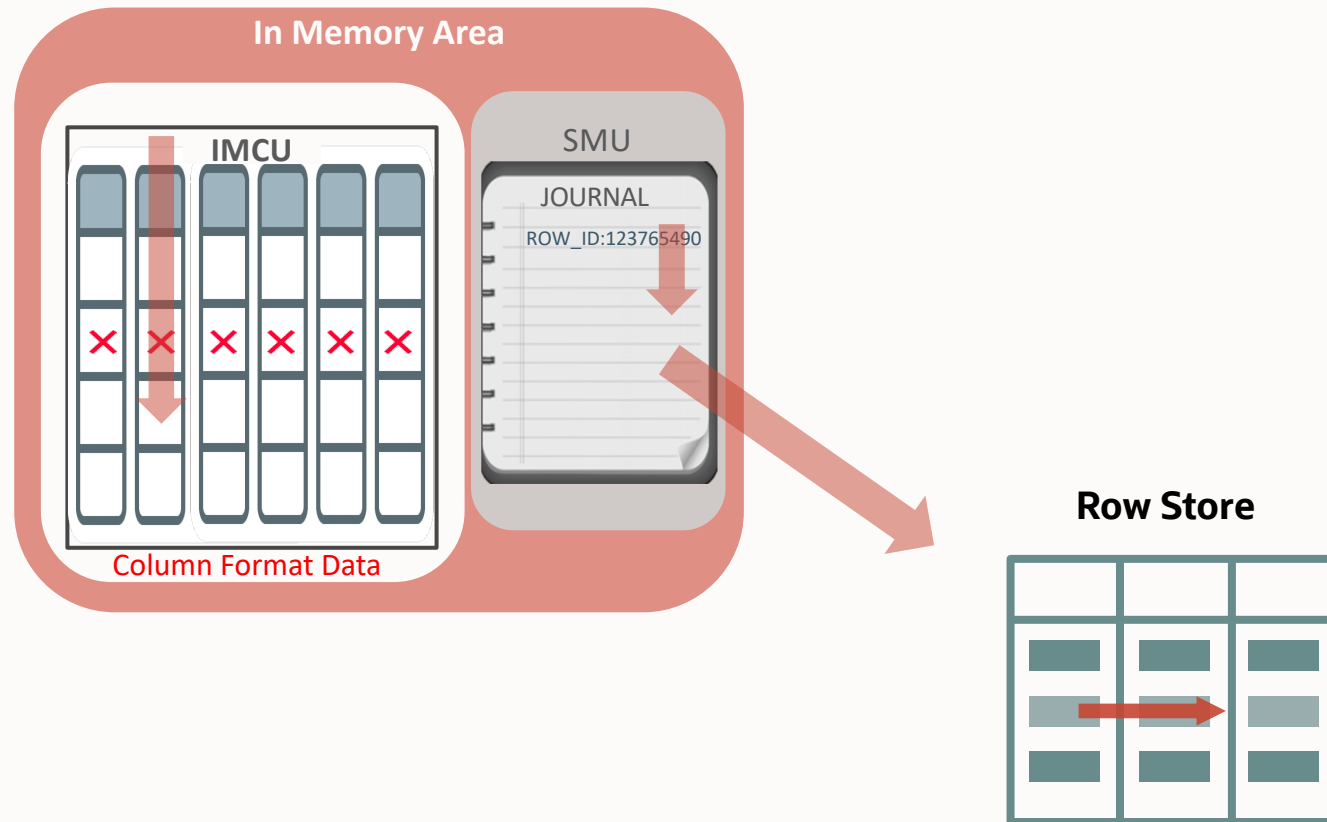
How Does Database In-Memory Handle Data Changes?

DML and the In-Memory Column Store



- DML operations processed in row store just as they are today
- Corresponding entry in column store marked stale as of SCN
- ROWID of row stored in the Transaction Journal

DML and the In-Memory Column Store

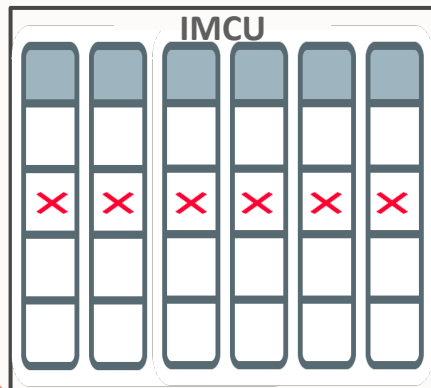


- In-Memory Column Store is never out of date
- Read-consistency is achieved by merging contents of column, the transaction journal and buffer cache
- Pure In-Memory format means no undo/redo to repopulate IMCUs

Repopulation and the In-Memory Column Store

In Memory Area

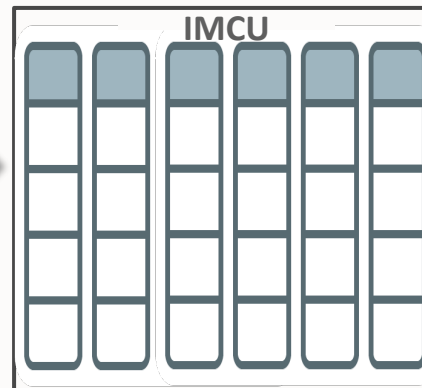
- 1 Mark original IMCU as old version as of SCN



Original IMCU

- 2 Create a new version of IMCU by combining clean rows from original with latest version of changed rows

- 3 Track any DML operations that occur during IMCU creation in Journal



New version IMCU

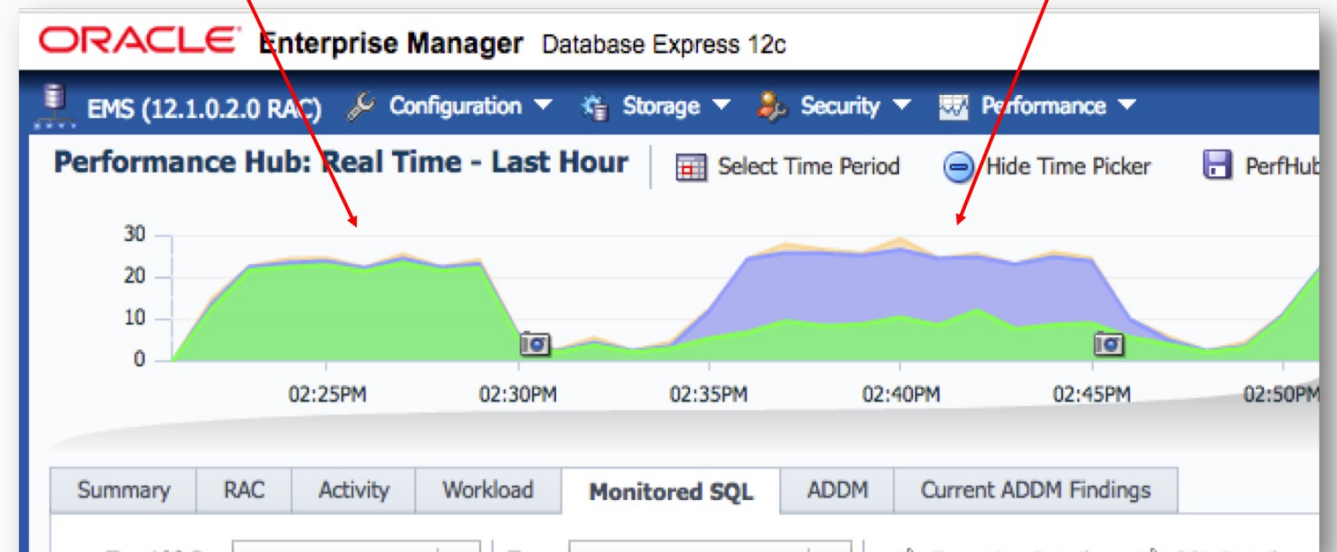
Both versions of the IMCU will exist in IM column store until original is no longer useful or there is space pressure

Real World Application Demo – DBIM YouTube Channel

- Database In-Memory enables real time analytics
- OLTP transaction workload with Database In-Memory
- Workload is not allowed to exceed the capacity of the machine
 - OLTP workload (simulates 6400 users each running a transaction every 10 seconds)
 - OLAP workload (20 connections running a stream of 100 randomly selected reports with no think time)

OLAP (IM Enabled) + OLTP

OLAP (IM Disabled) + OLTP



DBIM enabled – Maximum throughput, no OLTP degradation

DBIM disabled – I/O required, fewer reports, OLTP impacted

What's New in Database In-Memory?

Database In-Memory Innovations

21c

- **Self Managing In-Memory**
- In-Memory Spatial Analytics
- In-Memory Full Text Columns
- External Table Enhancements
- Hybrid Scans
- JSON Data Type
- Vector Joins
- Base Level Feature

19c

- Performance
- External Tables: Hive & HDFS
- Memoptimized Rowstore – Fast Ingest

18c

- **Automatic In-Memory**
- In-Memory Dynamic Scans
- In-Memory External tables
- In-Memory Optimized Arithmetic
- Memoptimized Rowstore – Fast Lookup

12.2

- Join Groups
- In-Memory Expressions
- JSON/OSON support
- **Massive capacity - In-Memory on Exadata flash**
- Auto population policies
- Fast-Start
- Active Data Guard

12.1

- Pure In-Memory column format
- Scan & Filter on compressed data
- Fast joins
- Data pruning via storage indexes
- SIMD vector processing
- In-Memory aggregation

21c

Self-Managing, Convergence

19c

Performance, Automation

18c

Performance, Capacity

12.2

12.1

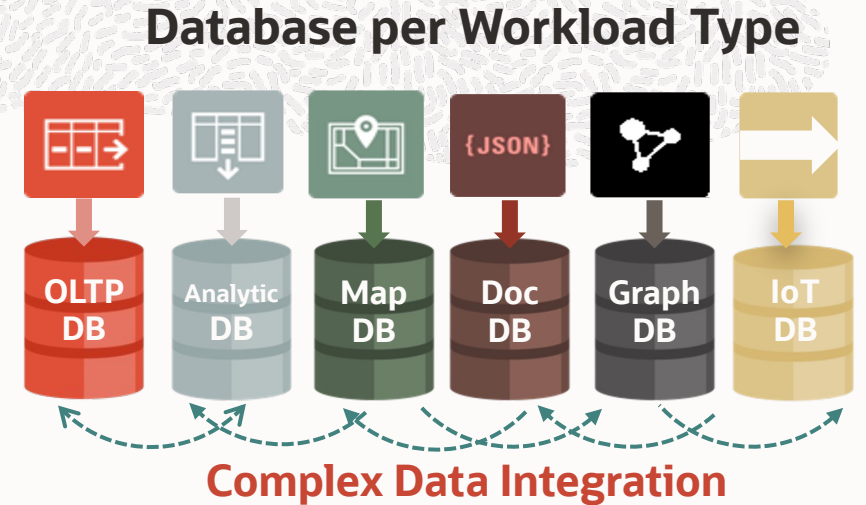


Converged Oracle Database

Oracle Converged Database

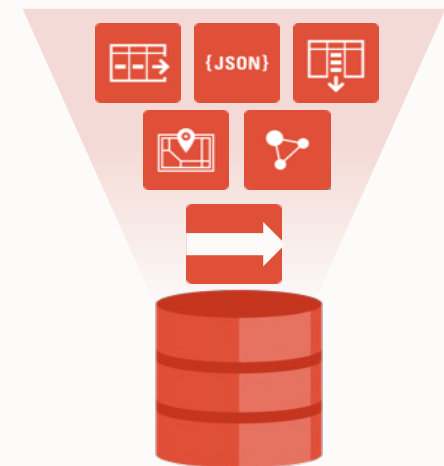
Path to Simpler Application Evolution

- As applications evolve, they often need other algorithms: Document, Graph, AI/ML. etc.
- Oracle converged database enables any type of workloads against any type of data
 - Relational, Document, Graph, Spatial, Files, etc.
 - Class leading Machine Learning portfolio
- Converged DB is: **Simpler, Lower Cost, More Reliable**
 - Same **security** management across all data
 - Prevents **data fragmentation** and **copy contagion**
 - Eliminates **multi-database integration costs**



V.S.

Converged DB for **All** Workloads



No Data Movement!



Converged Oracle Database

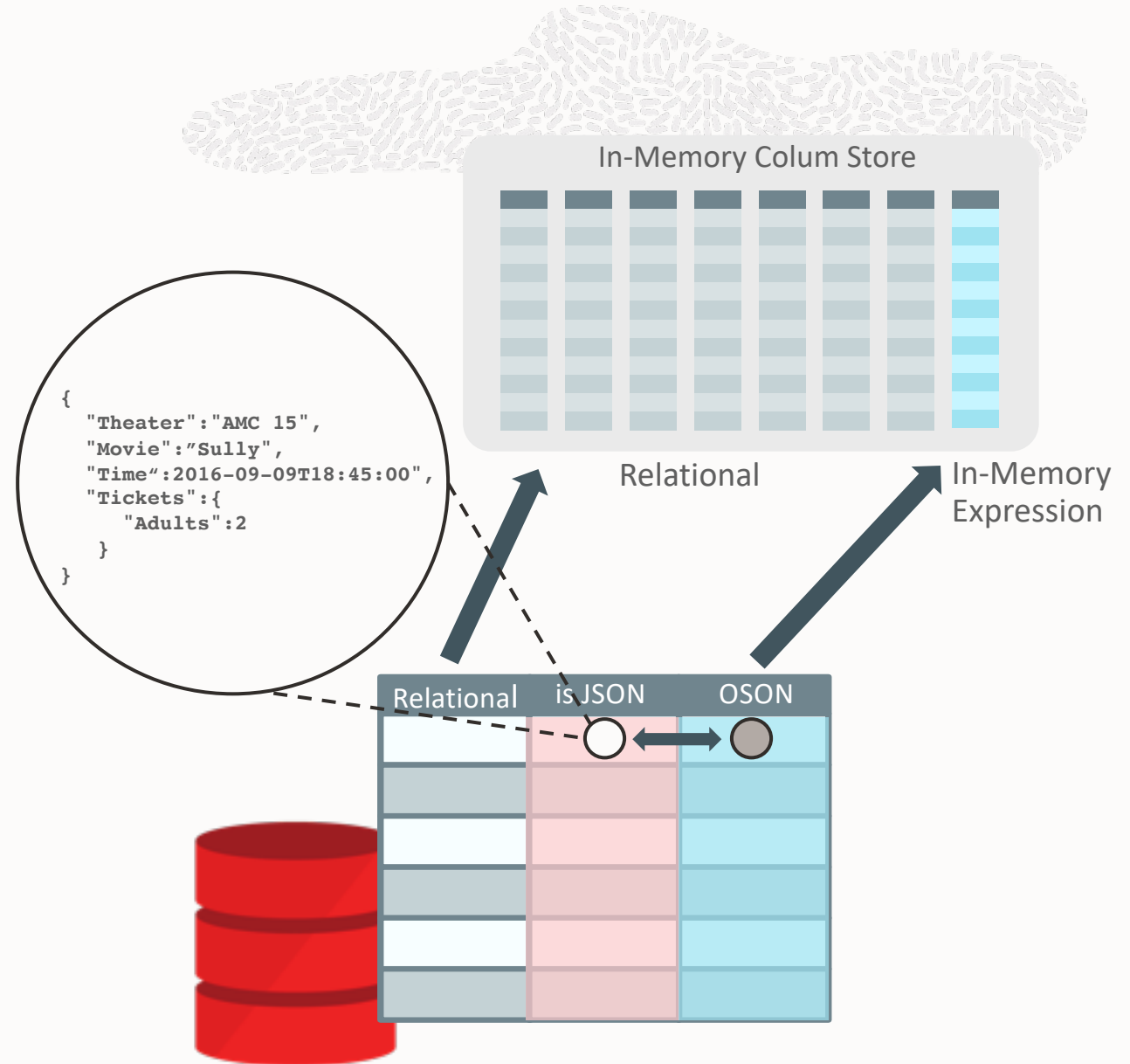
JSON with Database In-Memory

Using JSON with Database In-Memory

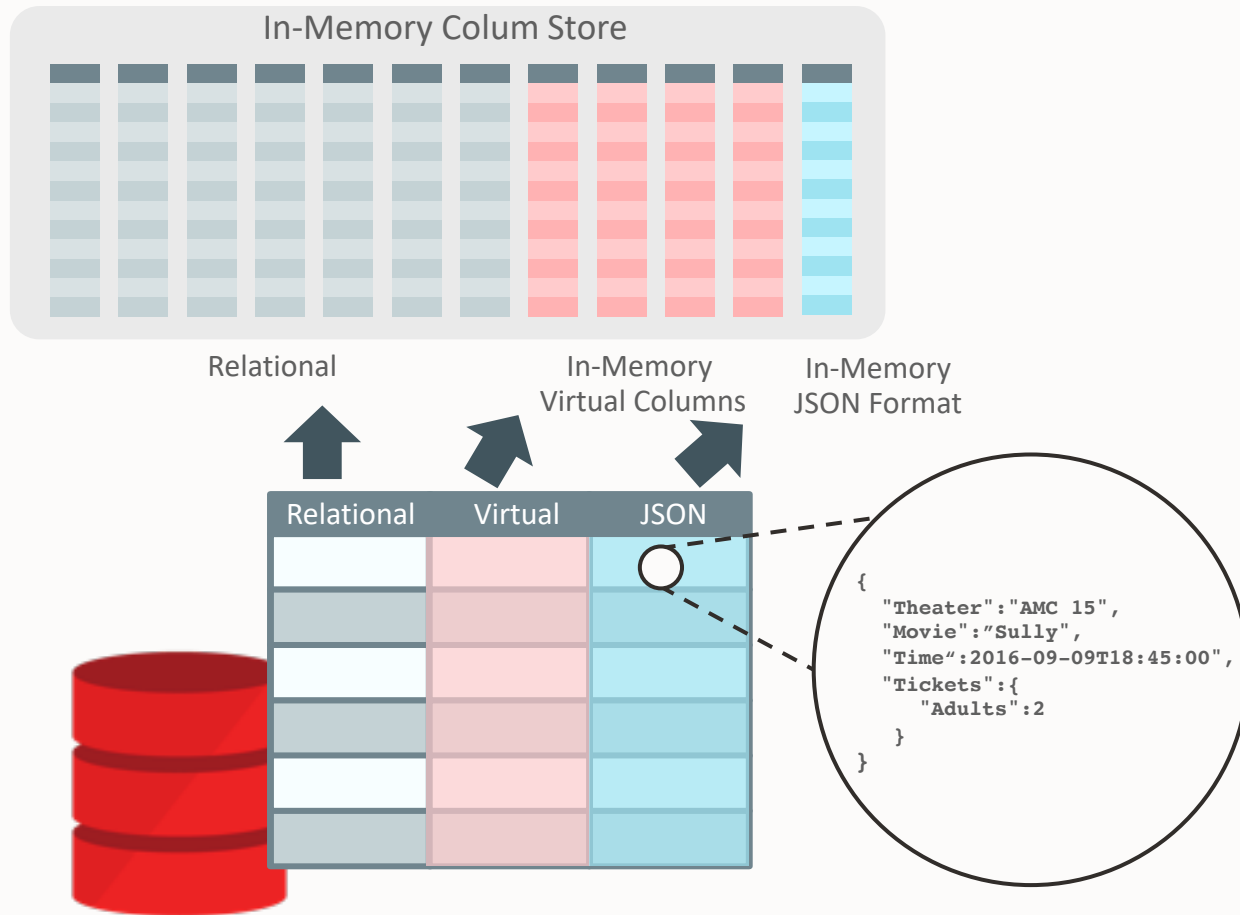
- Database compatibility set to 12.2.0.0 or higher
- max_string_size must be set to 'extended'
 - 32K VARCHAR2 columns can be populated in the IM column store
 - See blog post: [Storing Values Up To 32KB In Size In The In-Memory Column Store](#)
- inmemory_expressions_usage set to STATIC_ONLY or ENABLE
- inmemory_virtual_columns set to ENABLE
- JSON data columns must have 'is json' check constraints
- Database In-Memory supports a special "binary" JSON format called **OSON** that performs better than row-oriented JSON

In-Memory JSON

- JSON documents populated as an In-Memory Expression using an optimized binary format (OSON)
- Additional expressions can be created on JSON columns (e.g. JSON_VALUE) & stored in column store
- Queries on JSON content or expressions automatically directed to In-Memory format
 - e.g. Find movies where movie.name contains 'Jurassic'
- **2X to 30X** performance gains observed



In-Memory JSON Data Type



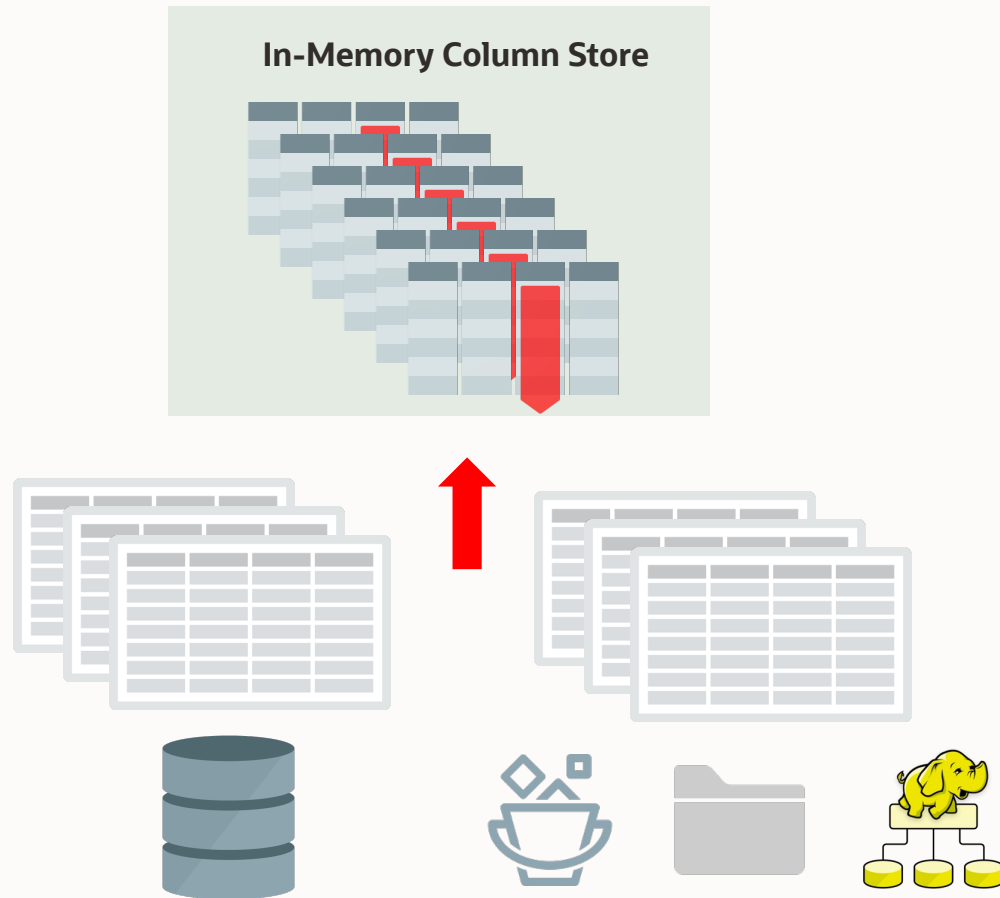
- New JSON data type columns populated up to 8KB inline using an optimized binary format
- IM queries on JSON content supported for `JSON_TABLE`, `JSON_VALUE`, `JSON_EXISTS` and `JSON_TEXTCONTAINS`
 - e.g. Find movies where movie.name contains "Jurassic"
- Additional expressions can be created on JSON columns (e.g. `JSON_VALUE`) & stored in column store
- **2X to 30X** performance gains observed

Converged Oracle Database

Other Data Supported with Database In-Memory



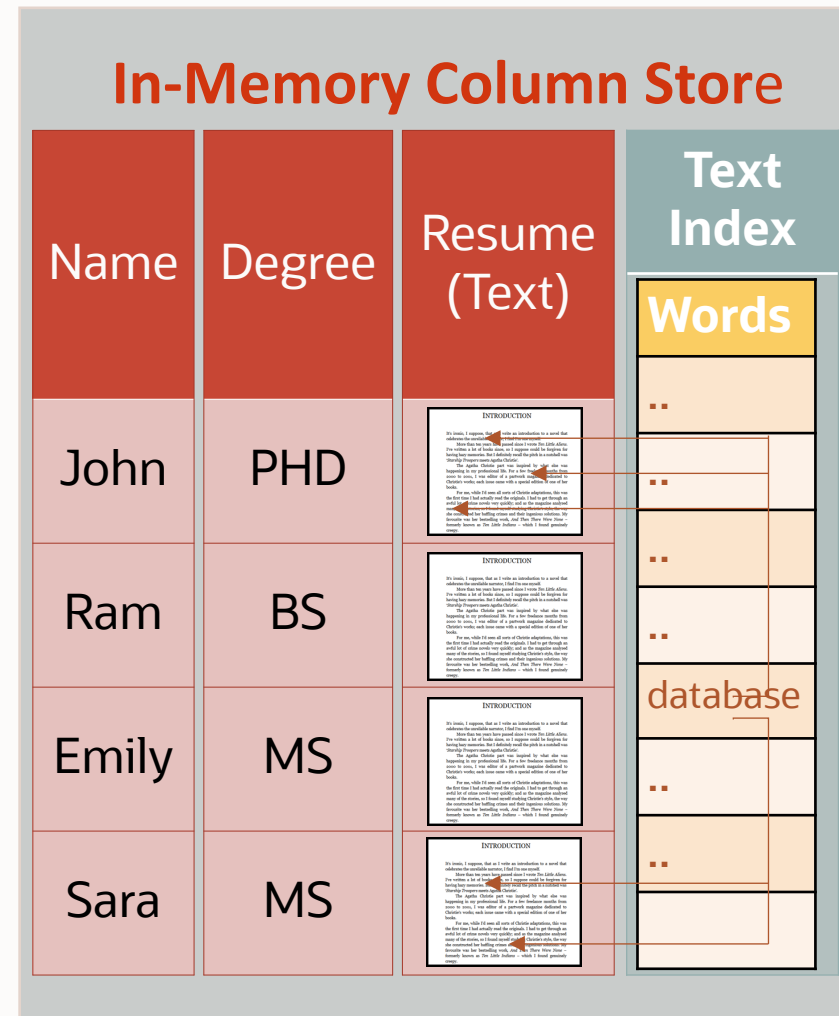
Enhanced In-Memory External Table Support



- Now supports population for partitioned external tables or hybrid partitioned tables
 - INMEMORY clause at the top-level is inherited by every partition
 - INMEMORY attribute can also be specified for individual partitions
- Flexibility to only populate selected external and/or internal partitions

In-Memory Full Text Columns

- New In-Memory only Inverted Index for each text column
- Inverted Index maps words to documents containing those words
- Converged queries (*relational + text*) faster since executed as IM table scans
- Replaces on-disk text index for analytic workloads
- Text Queries run **3x faster**



Find job candidates with "PhD" degrees who have "database" in their resumes

In-Memory Spatial

New In-Memory only Spatial Summary column for each spatial column

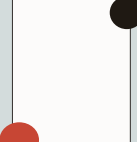
- Spatial Summary - compact approximation of spatial details

Spatial Summaries stored in In-Memory formats

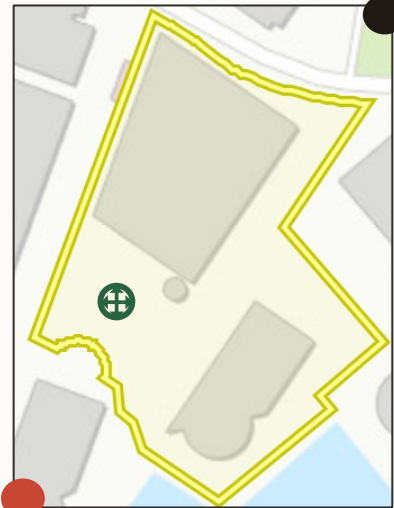
- Filter values using SIMD vector scans
- Replace R-Tree Indexes for searches

Spatial Queries up to **10x** faster

- No analytic R-tree index maintenance needed

In-Memory (IM) Table Columns			Additional IM columns
Parcel Number	Parcel Address	Spatial Details	Spatial Summary
095040390	300 Oracle Pkwy		
095040310	400 Oracle Pkwy		
095040250	500 Oracle Pkwy		
095040260	600 Oracle Pkwy		

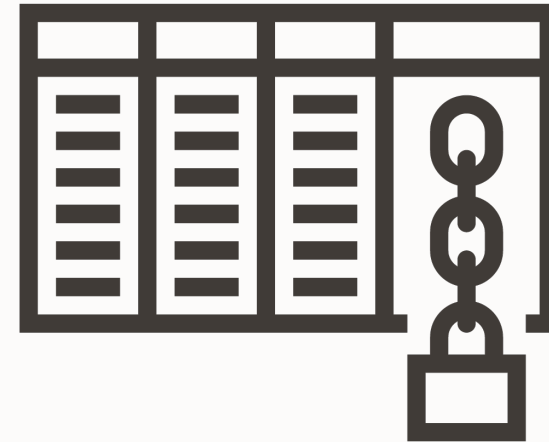
Search 140 Million US land parcels



Which parcel is the utility valve in?

In-Memory and Blockchain Tables

- Database In-Memory fully supports Blockchain tables
- See Ask TOM Office Hours session – [Database In-Memory and Blockchain tables](#)



Database In-Memory Base Level Feature

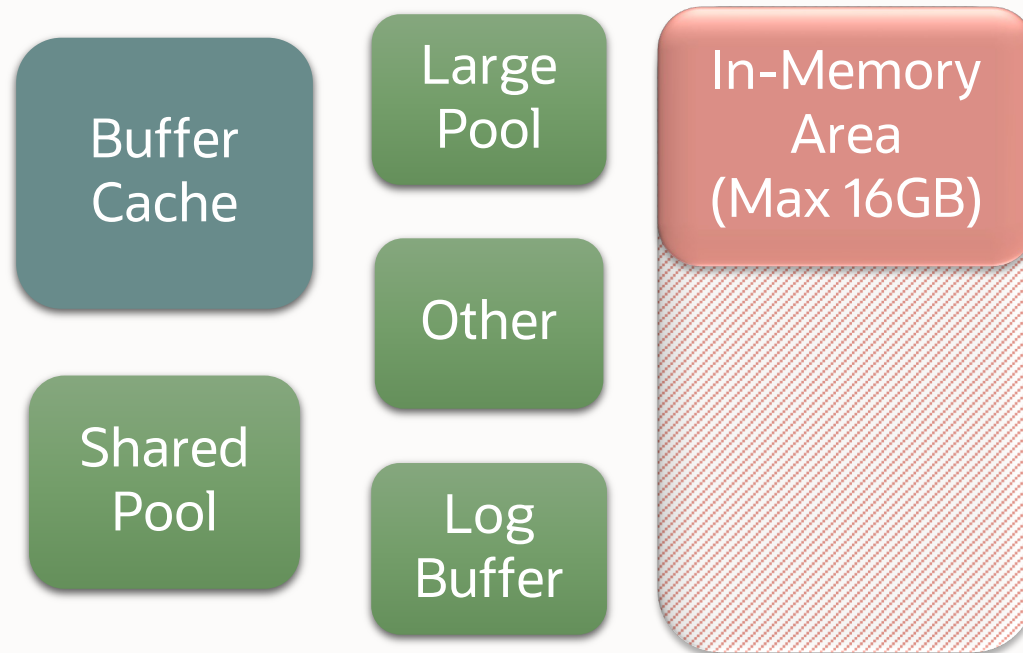
Database In-Memory Base Level Feature

- Customers can now use up to a 16GB IM column store without having to license the Database In-Memory option
- The purpose of this Base Level feature is to allow customers to see the value of Database In-Memory
- Not all Database In-Memory features are available with Base Level
- Available in 21c and starting with the 19.8 RU
- No backports to previous versions are planned
- See [Database In-Memory AskTOM Office Hours sessions](#) and [blog posts](#)

Base Level Feature



System Global Area (SGA)



- Enabled with INMEMORY_FORCE parameter
 - Must be set to BASE_LEVEL
- INMEMORY_SIZE limited to a maximum of 16GB
- Feature tracking reports usage as "In-Memory Base Level"

Where Can You Get More Information?

https://blogs.oracle.com/in-memory/dbim-resources

